

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО»
Факультет інформатики та обчислювальної техніки
(повна назва інституту/факультету)

Кафедра інформатики та програмної інженерії
(повна назва кафедри)

«До захисту допущено»

Завідувач кафедри

_____ Едуард ЖАРІКОВ
(підпис) (ім'я прізвище)

“ ” _____ 2022 р.

Дипломний проєкт
на здобуття ступеня бакалавра
за освітньо-професійною програмою «Інженерія програмного забезпечення
комп'ютерних систем»
спеціальності «121 Інженерія програмного забезпечення»

на тему: Вебзастосунок для управління автосалоном

Виконав студент IV курсу, групи ІТ-81
(шифр групи)
Паньків Олександр Васильович
(прізвище, ім'я, по батькові) _____ (підпис)

Керівник старший викладач, Халус О.А.
(посада, науковий ступінь, вчене звання, прізвище та ініціали) _____ (підпис)

Консультант
з графічної
документації доцент, к.т.н., доц., Ліщук К. І.
(посада, науковий ступінь, вчене звання, прізвище та ініціали) _____ (підпис)

Рецензент професор кафедри ІСТ, д.е.н., проф., Шемаєв В.М.
(посада, науковий ступінь, вчене звання, прізвище та ініціали) _____ (підпис)

Засвідчую, що у цій дипломній роботі
немає запозичень з праць інших
авторів без відповідних посилань.

Студент _____
(підпис)

Київ –2022

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

Рівень вищої освіти – перший (бакалаврський)

Спеціальність – 121 Інженерія програмного забезпечення

Освітньо-професійна програма – Інженерія програмного забезпечення
комп'ютерних систем

ЗАТВЕРДЖУЮ

Завідувач кафедри

(підпис) Едуард ЖАРІКОВ
(ім'я прізвище)

“ ” _____ 2022 р.

ЗАВДАННЯ
на дипломний проєкт студенту

Паньківа Олександра Васильовича

(прізвище, ім'я, по батькові)

1. Тема проєкту Вебзастосунок для управління автосалоном

керівник проєкту Халус Олена Андріївна, ст. викладач

(прізвище, ім'я, по батькові, науковий ступінь, вчене звання)

затверджені наказом по університету від «03» червня 2022 р. №877-с

2. Термін подання студентом проєкту « 21 » червня 2022 року

3. Вихідні дані до проєкту: технічне завдання

4. Зміст пояснювальної записки

1) Аналіз вимог до програмного забезпечення: загальні положення, огляд існуючих рішень, аналіз вимог до програмного забезпечення.

2) Моделювання та конструювання програмного забезпечення: моделювання та аналіз програмного забезпечення, проєктування бази даних, вибір стеку технологій, конструювання програмного забезпечення, аналіз безпеки даних.

3) Аналіз якості і тестування програмного забезпечення: аналіз якості ПЗ, опис процесу тестування, контрольний приклад, системні вимоги до ПЗ, апаратні вимоги до ПЗ.

4) Впровадження та супровід програмного забезпечення: розгортання програмного забезпечення, робота з програмним забезпеченням.

5. Перелік графічного матеріалу

1) Схема бази даних _____

2) Схема структурна компонентів програмного забезпечення _____

3) Креслення вигляду екранних форм _____

6. Консультанти розділів проєкту

Розділ	Прізвище, ініціали та посада консультанта	Підпис, дата	
		завдання видав	завдання прийняв

7. Дата видачі завдання «10» березня 2022 року _____

Календарний план

№ з/п	Назва етапів виконання дипломного проєкту	Термін виконання етапів проєкту	Примітка
1	Затвердження теми роботи	10.03.2021	
2	Вивчення предметної області	11.04.2021	
3	Аналіз існуючих рішень	14.04.2021	
4	Вибір та ознайомлення з середовищем розробки	17.04.2021	
5	Вибір СУБД	20.04.2021	
6	Проектування БД	25.04.2021	
7	Моделювання застосунку	27.04.2021	
8	Розробка застосунку	29.04.2021	
9	Перевірка функціонування застосунку	03.05.2021	
10	Виконання графічних матеріалів	19.05.2021	
11	Оформлення пояснювальної записки	23.05.2021	
12	Подання ДП на попередній захист	25.05.2021	
13	Подання ДП рецензенту	09.06.2021	
14	Подання ДП на основний захист	21.06.2022	

Студент

_____ (підпис)

Олександр ПАНЬКІВ

_____ (ініціали, прізвище)

Керівник

_____ (підпис)

Олена ХАЛУС

_____ (ініціали, прізвище)

АНОТАЦІЯ

Пояснювальна записка дипломного проекту складається з чотирьох розділів, містить 29 таблиць, 18 рисунків, 8 джерел та 3 додатки – загалом 79 сторінок.

Дипломний проект присвячений розробці мікросервісного вебзастосунок для управління автосалоном.

Метою є створення зручного засобу для управління автосалоном.

Об'єкт дослідження: мікросервіси.

Предмет дослідження: вебзастосунок для управління автосалоном на основі архітектури мікросервісів.

У першому розділі розглянуто існуючі рішення та визначено вимоги до розроблюваного застосунку.

Розділ другий присвячений проектуванню бази даних, визначенню набору використовуваних технологій та конструювання програмного забезпечення з їх використанням.

В третьому розділі визначено вимоги якості програмного забезпечення та проведено тестування.

В четвертому розділі описано процеси розгортання компонентів програмного забезпечення.

КЛЮЧОВІ СЛОВА: ВЕБЗАСТОСУНОК, БАЗА ДАНИХ, МІКРОСЕРВІС, АВТОСАЛОН, АРХІТЕКТУРА, УПРАВЛІННЯ.

ABSTRACT

The explanatory note of the diploma project consists of four sections, contains 29 tables, 18 figures, 8 sources and 3 appendices – in total 79 pages.

The diploma project is dedicated to the development of a microservice-based web application for car dealership management.

The goal is to create a convenient tool for car dealership management.

Object of research: microservices.

Subject of research: web application for car dealership management based on microservice architecture.

The first section discusses the existing solutions and defines the requirements for the developed application.

The second section is dedicated to designing a database, defining the set of used technologies and designing software using them.

In the third section the software quality requirements were defined and testing conducted.

The fourth section describes the software deployment processes.

KEY WORDS: WEB APPLICATION, DATABASE, MICROSERVICE, CAR DEALERSHIP, ARCHITECTURE, MANAGEMENT.

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2022 р.

ВЕБЗАСТОСУНОК ДЛЯ УПРАВЛІННЯ АВТОСАЛОНОМ

Технічне завдання

КПІ.IT-7311.045440.01.91

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Олена ХАЛУС

Нормоконтроль:

_____ Катерина ЛІЩУК

Виконавець:

_____ Олександр ПАНЬКІВ

Київ – 2022

ЗМІСТ

1 НАЙМЕНУВАННЯ ТА ГАЛУЗЬ ЗАСТОСУВАННЯ	3
2 ПІДСТАВА ДЛЯ РОЗРОБКИ	4
3 ПРИЗНАЧЕННЯ РОЗРОБКИ	5
4 ВИМОГИ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	6
4.1 Вимоги до функціональних характеристик.....	6
4.2 Вимоги до надійності.....	6
4.3 Умови експлуатації	6
4.4 Вимоги до складу і параметрів технічних засобів.....	7
4.5 Вимоги до інформаційної та програмної сумісності.....	7
4.6 Вимоги до маркування та пакування	7
4.7 Вимоги до транспортування та зберігання.....	7
4.8 Спеціальні вимоги.....	7
5 ВИМОГИ ДО ПРОГРАМНОЇ ДОКУМЕНТАЦІЇ.....	8
6 СТАДІЇ І ЕТАПИ РОЗРОБКИ	9
7 ПОРЯДОК КОНТРОЛЮ ТА ПРИЙМАННЯ.....	10

					КПІ.ІТ-7311.045440.01.91	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		2

1 НАЙМЕНУВАННЯ ТА ГАЛУЗЬ ЗАСТОСУВАННЯ

Назва розробки: Вебзастосунок для управління автосалоном

Галузь застосування:

Наведене технічне завдання поширюється на розробку програмного забезпечення «Вебзастосунок для управління автосалоном» [045440], що застосовується для здійснення управління автосалоном та призначена для співробітників автосалонів.

					КПІ.ІТ-7311.045440.01.91	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		3

2 ПІДСТАВА ДЛЯ РОЗРОБКИ

Підставою для розробки вебзастосунку для управління автосалоном є завдання на дипломне проектування, затверджене кафедрою інформатики та програмної інженерії Національного технічного університету України «Київський політехнічний інститут імені Ігоря Сікорського».

					КПІ.ІТ-7311.045440.01.91	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		4

3 ПРИЗНАЧЕННЯ РОЗРОБКИ

Розробка призначена для покращення процесу управління автосалоном.

Метою розробки є створення мікросервісного вебзастосунку, що може слугувати предметом дослідження даного типу архітектури програмного забезпечення, а також для здійснення управління автосалоном.

					КПІ.ІТ-7311.045440.01.91	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		5

4 ВИМОГИ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

4.1 Вимоги до функціональних характеристик

4.1.1 Програмне забезпечення повинно забезпечувати виконання наступних основних функцій:

4.1.1.1 Для користувача:

- отримувати інформацію про співробітників;
- отримувати інформацію про автомобілі;
- отримувати інформацію про замовників.

4.1.1.2 Для адміністратора системи:

- переглядати дані автосалону;
- додавати дані автосалону;
- змінювати дані автосалону;
- видаляти дані автосалону.

4.1.2 Розробку виконати на платформах Django, Flask, React, RabbitMQ та Docker.

4.1.3 Додаткові вимоги:

Додаткові вимоги відсутні.

4.2 Вимоги до надійності

4.2.1 Передбачити контроль введення інформації.

4.2.2 Передбачити захист від некоректних дій користувача.

4.2.3 Забезпечити узгодженість даних в компонентах застосунку.

4.3 Умови експлуатації

4.3.1 Умови експлуатації згідно СанПін 2.2.2.542 – 96.

4.3.2 Обслуговування

Не висуваються.

					КПІ.ІТ-7311.045440.01.91	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		6

4.3.3 Обслуговуючий персонал

Не висуваються.

4.4 Вимоги до складу і параметрів технічних засобів

4.4.1 Програмне забезпечення повинно функціонувати на комп'ютерах з операційною системою Windows.

4.4.2 Мінімальна конфігурація технічних засобів:

4.4.2.1 Тип процесору core i3.

4.4.2.2 Об'єм ОЗП 8 ГБ.

4.4.2.3 Об'єм вільного дискового простору..... 5 ГБ

4.5 Вимоги до інформаційної та програмної сумісності

4.5.1 Програмне забезпечення повинно працювати під управлінням операційної системи Windows 10.

4.5.2 Вхідні дані повинні бути представлені в наступному форматі:
HTTP запит.

4.5.3 Результати повинні бути представлені в наступному форматі:
вебсторінка.

4.5.4 Програмне забезпечення повинно бути розроблене з використанням мови програмування python в середовищі розробки PyCharm.

4.6 Вимоги до маркування та пакування

Вимоги до маркування та пакування не пред'являються.

4.7 Вимоги до транспортування та зберігання

Вимоги до транспортування та зберігання не пред'являються.

4.8 Спеціальні вимоги

Згенерувати установчу версію програмного забезпечення.

					КПІ.ІТ-7311.045440.01.91	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		7

5 ВИМОГИ ДО ПРОГРАМНОЇ ДОКУМЕНТАЦІЇ

5.1 Програмні модулі, котрі розробляються, повинні бути задокументовані, тобто тексти програм повинні містити всі необхідні коментарі.

5.2 Програмне забезпечення повинно мати довідникову систему.

5.3 У склад супроводжувальної документації повинні входити наступні документи:

5.3.1 Пояснювальна записка не менше ніж на 50 аркушах формату А4 (без додатків 5.3.2 - 5.3.4).

5.3.2 Технічне завдання.

5.3.3 Керівництво користувача.

5.3.4 Програма та методика тестування

5.4 Графічна частина повинна бути виконана не менше ніж на 3 аркушах формату А3, котрі включаються у якості додатків до пояснювальної записки:

5.4.1 Схема бази даних.

5.4.2 Схема структурна компонентів програмного забезпечення.

5.4.3 Креслення вигляду екранних форм.

6 СТАДІЇ І ЕТАПИ РОЗРОБКИ

№	Назва етапу	Строк	Звітність
1	Затвердження теми роботи	10.03.2021	
2	Вивчення предметної області	11.04.2021	Опис проаналізованої предметної області
3	Аналіз існуючих рішень	14.04.2021	Опис існуючих рішень
4	Вибір та ознайомлення з середовищем розробки	17.04.2021	
5	Вибір СУБД	20.04.2021	Опис СУБД
6	Проектування БД	25.04.2021	Інфологічна, даталогічна моделі БД, схема бази даних
7	Моделювання застосунку	27.04.2021	Схема структурна компонентів програмного забезпечення
8	Розробка застосунку	29.04.2021	Тексти програмного забезпечення
9	Перевірка функціонування застосунку	03.05.2021	Тести, результати тестування
10	Виконання графічних матеріалів	19.05.2021	Графічний матеріал проекту
11	Оформлення пояснювальної записки	23.05.2021	Текст пояснювальної записки
12	Подання ДП на попередній захист	25.05.2021	
13	Подання ДП рецензенту	09.06.2021	
14	Подання ДП на основний захист	21.06.2022	

7 ПОРЯДОК КОНТРОЛЮ ТА ПРИЙМАННЯ

Тестування розробленого програмного продукту виконується відповідно до “Програми та методики тестування”.

					КПІ.ІТ-7311.045440.01.91	Арк.
						10
Змін.	Арк.	№ докум.	Підп.	Дата.		

**Пояснювальна записка
до дипломного проєкту**

на тему: Вебзастосунок для управління автосалоном

КПІ.ІТ-7311.045440.02.81

Київ – 2022

ЗМІСТ

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ.....	4
ВСТУП	5
1 АНАЛІЗ ВИМОГ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	8
1.1 Загальні положення	8
1.2 Огляд існуючих рішень.....	10
1.2.1 Automotive Creatio	10
1.2.2 Альфа-Авто	14
1.3 Аналіз вимог до програмного забезпечення	17
1.3.1 Розроблення функціональних вимог	17
1.3.2 Розроблення нефункціональних вимог	23
1.4 Висновки до розділу.....	23
2 МОДЕЛЮВАННЯ ТА КОНСТРУЮВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	25
2.1 Моделювання та аналіз програмного забезпечення	25
2.2 Проектування бази даних	26
2.2.1 Аналіз предметної області	26
2.2.2 Побудова моделі предметної області.....	27
2.2.2.1 Виділення сутностей	27
2.2.2.2 Побудова ER-діаграми	29
2.2.3 Даталогічне проектування.....	30
2.2.3.1 Формалізація зв'язків.....	30
2.2.3.2 Введення обмежень цілісності	31
2.2.3.3 Нормалізація бази даних.....	31
2.2.3.4 Визначення типів даних.....	32
2.3 Вибір стеку технологій	34
2.4 Конструювання програмного забезпечення	38
2.4.1 Адміністративний сервіс	38
2.4.2 Основний сервіс	44

2.4.3	Брокер повідомлень	51
2.4.4	Клієнтський компонент	54
2.5	Аналіз безпеки даних	58
2.6	Висновки до розділу.....	58
3	АНАЛІЗ ЯКОСТІ І ТЕСТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ ..	60
3.1	Аналіз якості ПЗ	60
3.2	Опис процесу тестування.....	60
3.3	Контрольний приклад	61
3.3.1	Модульний тест	61
3.3.2	Тест API.....	62
3.3.3	Тест повідомлень.....	62
3.3.4	Тест узгодженості даних	63
3.3.5	Тест користувацького інтерфейсу	63
3.4	Системні вимоги до ПЗ	64
3.5	Апаратні вимоги до ПЗ	64
3.6	Висновки до розділу.....	65
4	ВПРОВАДЖЕННЯ ТА СУПРОВІД ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ ..	66
4.1	Розгортання програмного забезпечення.....	66
4.1.1	Розгортання контейнера адміністративної частини	66
4.1.2	Розгортання контейнера основної частини	67
4.1.3	Розгортання клієнта	67
4.2	Робота з програмним забезпеченням.....	67
4.3	Висновки до розділу.....	68
	ВИСНОВКИ.....	69
	ПЕРЕЛІК ВИКОРИСТАНИХ ДЖЕРЕЛ.....	71
	ДОДАТОК А ЛІСТИНГ МОДЕЛЕЙ DJANGO-SERVICES	72
	ДОДАТОК Б ВМІСТ ФАЙЛУ URLS.PY DJANGO-SERVICES	74
	ДОДАТОК В ВМІСТ ФАЙЛУ «CONSUMER.PY» SERVICES «MAIN»	76

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ

API	– (Application Programming Interface)	Прикладний програмний інтерфейс
БД	– База даних	
СУБД	– Система управління базами даних	
REST	– (Representational State Transfer)	Передача репрезентативного стану
ER	– (Entity-relationship)	сутність-зв'язок
AMQP	– (Advanced Message Queuing Protocol)	Стандарт протоколу обробки повідомлень

					КПІ.ІТ-7311.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		4

ВСТУП

Архітектура мікросервісів – це доволі новий підхід в розробці програмного забезпечення і про нього з’являється дедалі більше публікацій в мережі Інтернет, проте й досі не визначено однозначно, як повинен проєктуватися застосунок, заснований на цьому принципі. На погляд автора, цю тему також не достатньо розкрито в навчальному курсі спеціальності, тому таким чином в тому числі було заповнено прогалину в навчанні.

Принцип мікросервісів несе в собі певні переваги, на які варто звернути уваги при плануванні розробки програмного засобу [1]:

- кожен сервіс несе в собі певний завершений набір функцій, може розроблятися окремо від інших, що в тому числі дозволяє легше розподіляти роботу між членами команди розробників;
- мікросервіси повинні працювати автономно, тому порушення роботи або проведення обслуговування елементів системи не повинно значним чином впливати на працездатність інших функцій застосунку;
- можливість поєднання різних технологій між собою. В тому числі – можливе застосування різних мов програмування, поєднання роботи різноспрямованих розробників. Це пов’язане з тим, що засоби комунікації між мікросервісами є спільними для різних технологій розробки: HTTP-виклики через API, брокери повідомлень;
- застосування різних технологій збереження даних. Принцип мікросервісів передбачає використання власних сховищ різними сервісами. Це дозволяє обрати для кожного з них ту систему управління базами даних, яка найбільше відповідатиме поставленим цілям;
- масштабованість – при необхідності розширення існуючої системи немає необхідності вносити зміни у всю систему, достатньо це зробити із відповідними її частинами або ж впровадити нові сервіси

					КПІ.IT-7311.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		5

та встановити їх комунікацію з іншими частинами програмного рішення;

- керованість – дозволяє відлагоджувати кожен функцію застосунку, не зачіпаючи інші та не вдаючись в подробиці функціонування великої монолітної системи;
- повторне використання – так як мікросервіси – це готові самостійні засоби, їх реалізацію можна з мінімальними змінами застосовувати при розробці інших продуктів.

Метою розробки є створення вебзастосунку на основі архітектури мікросервісів, що може бути застосований в роботі автосалону та як предмет дослідження.

Задачі теоретичного та практичного спрямування, що повинні бути виконані для досягнення поставленої мети:

- вибір та вивчення засобів розробки програмного забезпечення на достатньому рівні: Python, Django, Flask, PyCharm, MySQL, HTML, JavaScript, React, RabbitMQ;
- засвоєння принципів проєктування реляційних баз даних;
- ознайомлення з принципами створення застосунків на основі мікросервісної архітектури;
- проєктування та розробка бекенду вебзастосунку;
- розгортання та налагодження розробленої реалізації із використанням Docker;
- обмін повідомленнями між сервісами вебзастосунку з використанням RabbitMQ
- розробка фронтенду проєкту.

Для розробки було обрано найбільш нові та актуальні версії технологій на момент початку роботи над дипломним проєктом.

Результатом роботи є вебзастосунок, спроектований на основі архітектури мікросервісів. Підхід, використаний при розробці, можна вивчати

					КПІ.IT-7311.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		6

на прикладі розробленого рішення. Це допоможе в створенні подібних програмних засобів у майбутньому.

Розроблене програмне рішення має потенціал для проведення подальших досліджень: як посібник українською мовою для ознайомлення із принципами розробки застосунків такого типу та як реальний приклад роботи. Також отриманий програмний продукт може застосовуватися для управління автосалоном.

					КПІ.ІТ-7311.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		7

1 АНАЛІЗ ВИМОГ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

1.1 Загальні положення

Розвиток програмного забезпечення за останні кілька років має чітку тенденцію: зменшується частка класичних прикладних програм для комп'ютерів серед усіх розробок. Консольні застосунки, які були витіснені раніше, залишаються здебільшого в якості утиліт для серверів. В той же час, серед нових розробок зростає частка мобільних прикладних програм та вебзастосунків. Такий склад справ обумовлений, передусім, завдяки поширенню:

- доступу до Інтернету серед населення;
- володіння смартфонами;
- пристосування підприємницької діяльності до нових вимог щодо функціонування бізнесу, зокрема – взаємодії з клієнтами.

У зв'язку з популярністю стільникових телефонів, провідні виробники даного типу пристроїв отримали змогу розширити виробництво. Як наслідок, більші обсяги продажу принесли більше прибутків. Це, в свою чергу, дозволило розширити виробництво, проводити дослідження та впроваджувати нові розробки. В кінцевому результаті сучасний мобільний телефон не тільки надає доступ до стільникового зв'язку, а й є смартфоном, тобто частково виконує функції персонального комп'ютера. Однією з найважливіших із них є під'єднання до Всесвітньої мережі.

Як наслідок масової комп'ютеризації та поширення смартфонів, надавачі послуг Інтернету зіткнулися з попитом, що зростає. Кількість абонентів, що бажали отримати доступ до Всесвітньої мережі, збільшувалася. Мережеве обладнання також розвивалося. Як наслідок, послуги з надання доступу до Інтернету стали дешевшими і більш доступними для споживачів.

Підприємці почали створювати вебсайти для розширення взаємодії з клієнтами. Такий підхід дозволяє продемонструвати асортимент товарів та

					КПІ.IT-7311.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		8

послуг потенційному клієнтові. Також таким чином налагоджується зв'язок іще одним способом, окрім, до прикладу, дзвінків, листування чи зустрічі віч-на-віч. З'явилися Інтернет-магазини, згодом дедалі більше підприємств додали можливість купівлі та розрахунку через Всесвітню мережу.

На сьогодні споживач послуг очікує, що надавач буде представлений в Інтернеті. Це включає в себе не тільки вебсайти, але й сторінки в соцмережах, а також боти та канали в месенджерах. Підприємець, що не відповідає сучасним вимогам, програватиме своїм конкурентам.

Попит на створення вебсайтів обумовив швидкий розвиток відповідних засобів та технологій розробки. Було вдосконалено, а також винайдено нові підходи. Серед іншого, з'явився сучасний архітектурний стиль – мікросервіси.

Попри те, що даний підхід до розробки програмного забезпечення не є надто новим, він досі не отримав значного поширення. Також, хоча в освітніх програмах закладів вищої освіти розглядається даний архітектурний стиль в теорії, практичні варіанти його реалізації є недостатньо розкритими.

Таким чином, було прийнято рішення розробити вебзастосунок, заснований на архітектурі мікросервісів. Результати роботи можна буде застосовувати як наочний навчальний матеріал. Також, це послугує базою для подальших досліджень в даній сфері.

В якості предметної області розглядалися такі типові варіанти, як:

- онлайн-магазин;
- застосунок для ресторану;
- застосунок для складу;
- чат-бот в месенджері;
- автосалон.

З огляду на значну поширеність інших опцій, було вирішено обрати саме автосалон. Незвичний вибір предметної області допоміг забезпечити оригінальність підходу до розробки.

1.2 Огляд існуючих рішень

Внаслідок пошуку в мережі Інтернет було виявлено брак програмних засобів в обраній предметній області. Автосалон є дещо специфічним видом підприємницької діяльності. Набагато більш поширеними є станції технічного обслуговування, тому ринок програмного забезпечення більше спрямований саме на цей сектор автомобільної галузі.

Значною перепоною в дослідженні стала відсутність застосунків з відкритим вихідним кодом. Тому конкретну реалізацію програмного продукту довелося розробляти власноруч.

1.2.1 Automotive Creatio

Automotive Creatio – система повного циклу для автосалонів та дилерів автомобілів: від приваблення та першої консультації, до візиту в салон і здійснення покупки [2]. Серед переваг, які обіцяють: управління базою автомобілів, збільшення кількості відвідувань і середнього чеку замовлення, надання бездоганного клієнтського сервісу з гнучкими механіками від Creatio. Завдяки необмеженим можливостям BPM-системи і CRM-інструментів на єдиній платформі ви зможете консолідувати всі необхідні сервіси і сторонні застосунки для управління повною подорожжю клієнта: від приваблення, до обслуговування і повторних продажів. Сторінку продукту автосалону наведено на рисунку 1.1

					КПІ.IT-7311.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		10

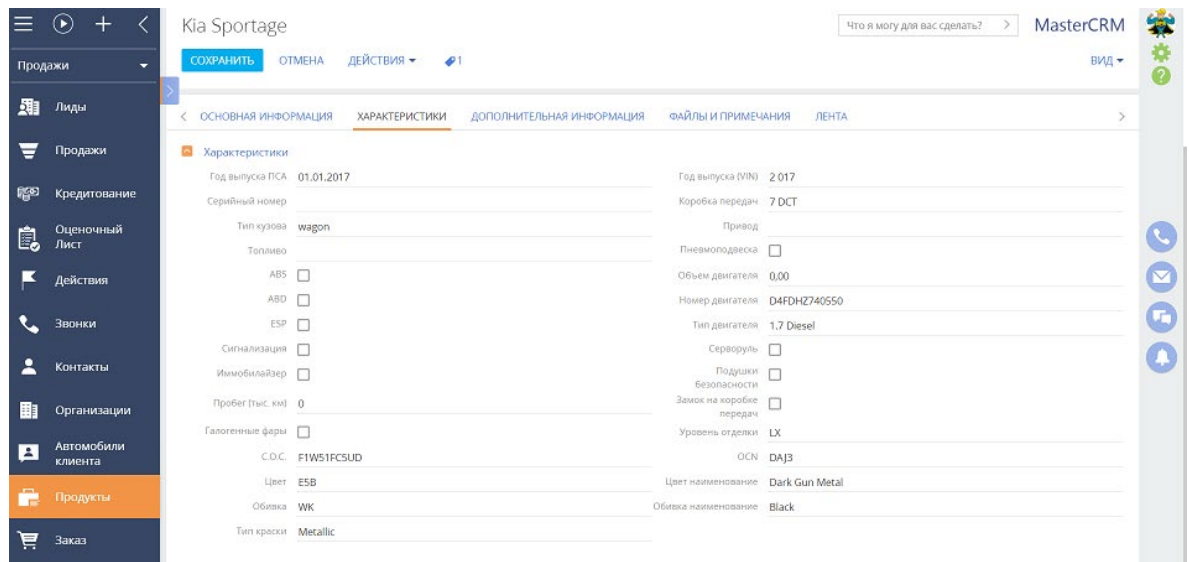


Рисунок 1.1 – Сторінка продукту автосалону

Єдина клієнтська база. Консолідування відомостей про наявних та потенційних клієнтів в єдиній базі даних:

- формування повного клієнтського профілю;
- глибока сегментація бази контактів та контрагентів для визначення цільової аудиторії;
- повна історія взаємовідносин: візити до салону, тест-драйви, консультації, покупки, сервісні звернення з прив'язкою до супутньої документації та рахунків;
- інструменти інтелектуального збагачення даних;
- дедуплікація інформації.

Управління продажами. Використання еталонних процесів Creatio для управління продажами:

- управління крос-продажами;
- планування KPI, аналіз ефективності співробітників, контроль виконання планів продажу;
- ведення робочого листування, призначення і контроль зустрічей, дзвінків, активностей;
- фіксація дисконтів та спеціальних умов продажу;

- ведення комерційної документації, рахунків, взаєморозрахунків з клієнтами.

Ведення каталогу автомобілів. Легке управління всіма продуктами компанії незалежно від складності та величини каталогу товарів:

- багаторівневий каталог продуктів;
- облік автомобілів та їх індивідуальних характеристик: VIN-номера, комплектації, стану, реєстраційних даних;
- сервісна історія автомобіля: інформація про кількість власників, проведені роботи в рамках гарантійного та післягарантійного обслуговування, переобладнання, проходження техобслуговування;
- фасетний підбір продуктів;
- імпорт каталогу і прайс-листів з інших систем.

Управління бізнес-процесами. Використання найкращих практик для максимальної ефективності виконання процесів:

- управління лідами та продажами;
- автоматизація роботи з рахунками та замовленнями;
- готові процеси обслуговування;
- візуальний дизайнер процесів і кейсів;
- бібліотека готових процесів;
- журнал виконання бізнес-процесів для моніторингу ситуації в реальному часі.

Підвищення ефективності маркетингу. Визначення та нарощування потреб клієнтів за допомогою потужних інструментів Creatio:

- управління маркетинговими кампаніями і активностями, планування бюджетів;
- готові процеси приваблення, розвитку та утримання;
- сегментація цільової аудиторії за територіальною ознакою, здійсненими покупками, потребами, фінансовими можливостями та іншими попередньо налаштованими параметрами;

					КПІ.IT-7311.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		12

- управління взаємовідносинами із підрядчиком та партнерами;
- аналіз джерел надходження лідів.

Управління сервісом. Використання готових процесів Creatio для бездоганного обслуговування:

- управління якістю та рівнем обслуговування, проведення опитувань задоволеністю та аналіз результатів;
- управління заказ-нарядами і контроль їх виконання;
- процес регулярних нагадувань для запрошення клієнта на технічне обслуговування або продовження контракту на обслуговування;
- планування відвідування клієнтами сервісу;
- планування і відстеженні завантаженні постів і співробітників станції технічного обслуговування;
- облік матеріалів та обладнання.

Управління документообігом. Зберігання всіх контрактів, а також пов'язаних з ними специфікацій та додаткових угод в єдиному реєстрі:

- єдина база стандартних форм і шаблонів документів;
- готовий процес візування контрактів;
- інструменти швидкого доступу до інформації за оплатою та рахунками;
- формування звітів та документів на основі попередньо налаштованих шаблонів.

Звітність та аналітика. Отримання інформації про ефективність своєї роботи в декілька натискань:

- аналітика за клієнтською базою;
- фінансові результати компанії, найбільш продавані моделі автомобілів;
- результати роботи менеджерів із продажу;
- підсумкова інформація про рівень задоволеності клієнтів;
- аналіз ефективності маркетингових впливів.

Вартість ліцензії становить 16240 гривень на рік.

1.2.2 Альфа-Авто

Програмний комплекс «Альфа-Авто», розроблений на платформі «1С:Підприємство 8», забезпечує комплексну підтримку всіх бізнес-процесів в центрах технічного обслуговування та автосалонах. Продукт призначений для автоматизації оперативної діяльності компаній у сфері автомобільного бізнесу [3].

Програма «Альфа-Авто» дозволяє організувати роботу складу, вести гуртову та роздрібну торгівлю запасними частинами, надавати послуги з ремонту та обслуговування автомобілів, оформлювати замовлення та продаж автомобілів, враховувати оплати і відстежувати стан взаєморозрахунків з покупцями та поставниками.

Користувачі програми мають можливість швидко формувати необхідні документи. Керівництво може вчасно отримувати і використовувати дані про різні аспекти діяльності компанії. Система надає інформацію, необхідну для прийняття управлінських рішень.

Завдяки архітектурі платформи «1С:Підприємство 8», документообіг «Альфа-Авто» адаптується під потреби реального підприємства. Приклад замовлення на автомобіль наведено на рисунку 1.2.

					КПІ.IT-7311.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		14

Заказ клиента на автомобиль № Ц000000001 от 24.10.2011 (Администратор) Проведен

Цены и валюта Действия Перейти (10:35:30) Оплата

Документ №: Ц000000001 от 24.10.2011 Автозапчасть; Центральная база Автозапч...; Администратор инфор...

№ заказа внутр.: Договор: Продажа от 21.10.11

Заказчик: Иванов Алексей Дмитриевич

Контрагент: Иванов Алексей Дмитриевич Статус: Поставлен Trade-In

Предоплата %: 50,00 USD: 10 000,00 Срок поставки: 31.10.2011

Банк: Рабочий лист:

Автомобиль Оpciones (0 поз.) Оборудование (0 поз.) Заказ-наряды Валюта: USD (30,0000) ИТОГО: 20 000,00

Модель: VOLKSWAGEN Sharan VR6 Цвет: Код ЛКП:

Комплектация: Комфорт Салон:

Автомобиль: VOLKSWAGEN Sharan VR6 VIN 6878790 VOLKSWAGEN Sharan VR6 VIN 687879075

Авто из наличия

Опция

Дизель 2.0

Автомат

Универсал

Цена автомобиля

Цена: 20 000,00 Ставка НДС: 18% Сумма НДС: 3 050,85

Скидка на авто: % скидки: 0,00 Сумма скидки: 0,00

Маркетинговая программа: Сумма: 20 000,00

Комментарий: Печать OK Записать Закрыть

Рисунок 1.2 – Замовлення на автомобіль

В застосунку реалізовано розподіл прав доступу на рівні користувачів, а також форм вводу, звітів, таблиць, записів. Завдяки цьому забезпечується надійний захист комерційної інформації компанії. Систему використовують понад 3000 організацій України та країн СНД.

«Альфа-Авто: Автосалон + Автосервіс + Автозапчастини, ред. 4» має в собі наступні облікові модулі:

- а) запчастини:
 - 1) закупки;
 - 2) роздрібна торгівля;
 - 3) гуртова торгівля;
 - 4) робота за замовленнями;
 - 5) організація руху товарів всередині фірми;
- б) автосервіс:
 - 1) планування ресурсів;
 - 2) оформлення ремонту;

- в) автосалон:
 - 1) замовлення на автомобілі;
 - 2) покупка та продаж автомобілів;
- г) фінансовий блок:
 - 1) оплати покупців та поставників;
 - 2) бюджетування;
- д) обмін даними:
 - 1) обмін даними з бухгалтерськими системами;
 - 2) обмін даними з каталогами виробників;
- е) єдина картотека транспортних засобів;
- ж) пошук клієнтів за різними даними: ідентифікаційний номер транспортного засобу, ПІБ, автомобільний номер;
- з) уніфікований, єдиний посібник адрес, що містить координати замовників, продавців, потенційних клієнтів та співробітників;
- и) збереження історій всіх клієнтів;
- к) можливість роботи менеджерів тільки зі своїми клієнтами;
- л) відстеження ефективності робіт менеджерів;
- м) відстеження стану замовлення запасних частин;
- н) сповіщення менеджерів про надходження номенклатури за замовленням покупця;
- о) простота попереднього перегляду всіх печатних документів з можливістю експорту до зовнішніх форматів: txt, xml, pdf, html;
- п) реєстрація упущеного попиту;
- р) можливість заборони продажів, здійснюваних нижче собівартості товару;
- с) відстеження стану документу для аналізу і контролю бізнес-операцій;
- т) підтримка декількох прайс-листів поставників;
- у) друк прайс-листів як за допомогою засобів «1С:Підприємство 8», так і за допомогою Microsoft Office;

- ф) клієнтські і складські замовлення на автомобілі;
- х) відповідальне зберігання автомобілів;
- ц) передпродажна підготовка автомобілів, установка додаткового оснащення.

Вартість ліцензії становить 24800 гривень.

1.3 Аналіз вимог до програмного забезпечення

Перед тим, як моделювати програмне забезпечення, необхідно визначити вимоги, що ставитимуться до нього. Для опису функціональних вимог зручно застосовувати схему структурну варіантів використання. Вона графічно відображає можливі випадки взаємодії різних користувачів із системою.

1.3.1 Розроблення функціональних вимог

Дійові особи (актори), що використовуватимуть вебзастосунок:

- адміністратор – має можливість виконувати операції над даними: читання, створення, зміна, видалення;
- переглядач – може тільки переглядати дані.

Варіанти використання переглядача описано в таблицях 1.1 – 1.4

Таблиця 1.1 – Варіант використання UC001

Назва	Перегляд автомобілів
Опис	Переглядач спостерігає повний список автомобілів
Учасники	Переглядач
Передумови	Переглядач переходить за посиланням на сторінку з автомобілями
Післяумови	Переглядач отримує вебсторінку, яка містить повний перелік автомобілів
Основний сценарій	Переглядач бажає переглянути автомобілі, які пропонує для замовлення автосалон та переходить за посиланням

Таблиця 1.2 – Варіант використання UC002

Назва	Перегляд співробітників
Опис	Переглядач спостерігає повний список співробітників автосалону
Учасники	Переглядач
Передумови	Переглядач переходить за посиланням на сторінку зі співробітниками
Післяумови	Переглядач отримує вебсторінку, яка містить повний перелік співробітників автосалону
Основний сценарій	Переглядачеві потрібно дізнатися, хто зі співробітників займає певні посади та отримати їхні контактні дані

Таблиця 1.3 – Варіант використання UC003

Назва	Перегляд посади
Опис	Переглядач спостерігає опис однієї з посад
Учасники	Переглядач
Передумови	Переглядач переходить за посиланням на сторінку з конкретною посадою
Післяумови	Переглядач отримує вебсторінку, яка містить інформацію про конкретну посаду
Основний сценарій	Переглядач отримав список співробітників. Оскільки посади позначені кодами, не одразу зрозуміло, що це саме за посада. Рядок з посадою є посиланням. Переглядач натискає на нього для отримання інформації про посаду співробітника, що його цікавить

Таблиця 1.4 – Варіант використання UC004

Назва	Перегляд виробника авто
Опис	Переглядач спостерігає інформацію про одного із виробників авто
Учасники	Переглядач
Передумови	Переглядач переходить за посиланням на сторінку з виробником авто
Післяумови	Переглядач отримує вебсторінку, яка містить усі дані про виробника авто
Основний сценарій	Переглядач отримав увесь список авто та одне з них зацікавило його. Рядок з виробником є посиланням. Переглядач натискає на нього з метою отримання даних про виробника (адреса, країна походження, опис)

Варіанти використання адміністратора описано в таблицях 1.5 – 1.*

Таблиця 1.5 – Варіант використання UC005

Назва	Перегляд замовників
Опис	Адміністратор отримує список замовників з даними про їх замовлення
Учасники	Адміністратор
Передумови	Адміністратор переходить за посиланням на адміністративну сторінку із замовниками
Післяумови	Адміністратор отримує вебсторінку, яка містить увесь перелік замовників та кнопки, що дозволяють здійснювати створення, зміну та видалення замовників
Основний сценарій	Адміністратор хоче вивести список замовників для того, щоб змінити дані одного з них. Він заходить до адміністративної частини застосунку та натискає на пункт меню «Замовники»

Таблиця 1.6 – Варіант використання UC006

Назва	Зміна даних замовника
Опис	Адміністратор змінює дані одного із замовників
Учасники	Адміністратор
Передумови	Адміністратор натискає на кнопку «редагувати» в адміністративному перегляді замовників
Післяумови	Атрибути одного з рядків таблиці «співробітники» бази даних змінюються відповідно до того, що написав адміністратор
Основний сценарій	Адміністраторові необхідно додати уточнювальну інформацію про замовника. Він натискає на кнопку «редагувати», змінює дані у відповідних полях та надсилає форму

Таблиця 1.7 – Варіант використання UC007

Назва	Додавання замовника
Опис	Адміністратор додає нового замовника
Учасники	Адміністратор
Передумови	Адміністратор натискає на кнопку «додати» в адміністративному перегляді замовників
Післяумови	В базу даних застосунку додається новий запис до таблиці «замовники»
Основний сценарій	Відвідувач автосалону оформлює замовлення на автомобіль. Адміністратор додає інформацію про замовника в систему

Таблиця 1.8 – Варіант використання UC008

Назва	Видалення замовника
Опис	Адміністратор видаляє замовника із системи
Учасники	Адміністратор
Передумови	Адміністратор натискає на кнопку «видалити» в адміністративному перегляді замовників
Післяумови	Із бази даних застосунку видаляється запис про відповідного замовника
Основний сценарій	Відвідувач автосалону в процесі оформлення замовлення змінив своє рішення та вирішив не вносити передплату. Адміністратор скасовує його замовлення, видаляючи замовника з системи

На основі описаних варіантів використання створено діаграму варіантів використання (рисунок 1.3).



Рисунок 1.3 – Схема структурна варіантів використання

Сформовано функціональні вимоги до програмного забезпечення.
Перелік зазначено в таблиці 1.9.

Таблиця 1.9 – Опис функціональних вимог

Ідентифікатор	Назва	Опис
REQ001	Перегляд даних	Програмне забезпечення дозволяє переглядати дані з БД системи у вигляді вебсторінок із вмістом
REQ002	Зміна даних	Програмне забезпечення надає можливість змінювати дані, що зберігаються в БД системи за допомогою користувацького інтерфейсу
REQ003	Додавання даних	Програмне забезпечення дозволяє додавати нові дані до БД системи за допомогою користувацького інтерфейсу
REQ004	Видалення даних	Програмне забезпечення повинне давати можливість видаляти дані, що зберігаються в БД системи. При цьому експлуатант взаємодіє із застосунком через користувацький інтерфейс

В матриці трасування відображено те, які варіанти використання яким функціональним вимогам відповідають (таблиця 1.10).

Таблиця 1.10 – Матриця трасування

	UC001	UC002	UC003	UC004	UC005	UC006	UC007	UC008
REQ001	*	*	*	*	*			
REQ002						*		
REQ003							*	
REQ004								*

1.3.2 Розроблення нефункціональних вимог

Вебзастосунок повинен бути розроблений з дотриманням стилю мікросервісів. Взаємодія користувача із системою відбувається через користувацький інтерфейс клієнта, реалізованого із застосуванням бібліотеки React. Він, в свою чергу, отримує дані, звертаючись REST-запитами до двох сервісів: «admin» та «main». Кожен з них надає свій API для взаємодії. В обох сервісів є своя незалежна БД під управлінням СУБД MySQL. Один з одним вони взаємодіють через брокер повідомлень RabbitMQ.

Сервіс «main» реалізовано з використанням python-мікрофреймворку Flask, а «admin» базується на фреймворку Django. Обидва сервіси розгортаються як ізольовані процеси у вигляді Docker-контейнерів.

Доступ до клієнта здійснюється через браузер. Для цього повинні бути придатні усі сучасні браузери, базовані на рушіях Gecko, WebKit або Blink.

Вебзастосунок повинен надавати інтуїтивно зрозумілий користувацький інтерфейс.

1.4 Висновки до розділу

Ознайомлення з існуючими рішеннями викликало труднощі. В першу чергу, це зумовлено тим, що всі знайдені засоби, які цілком відповідають поставленій меті, були платними та із закритим вихідним кодом. Існує значна кількість пропозицій щодо засобів для станцій технічного обслуговування. В той же час, хоч специфіка роботи частково перетинається із автосалоном, не було побачено достатнього фокусу на необхідних речах. При цьому ж деяка частина функціональності видалася надлишковою, як-то документообіг чи керування ремонтом. На думку автора роботи, для цього повинне бути передбачене окреме програмне забезпечення для бухгалтерського обліку та для роботи механіків: облік запчастин, завдань та планів, задля зосередження на специфічних цілях і уникнення громіздкості розроблюваного продукту.

					КПІ.IT-7311.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		23

Деякі засоби виглядали застарілими. Так як метою в процесі виконання дипломного проєкту значною мірою була дослідна робота, автор орієнтувався на застосування найсучасніших засобів розробки.

Кількість пропозицій в Україні дуже обмежена. Також недоліком автор вважає відсутність інтерфейсу українською мовою. Після розгляду наявних рішень було зроблено висновок про необхідність розробки власного варіанту програмного забезпечення для управління автосалоном. Проведено аналіз вимог до програмного забезпечення, розроблено функціональні та нефункціональні вимоги. Наявність відкритого вихідного коду дасть можливість у майбутньому розглядати розроблене рішення як приклад архітектури мікросервісів, а також, за потреби, розширювати можливості оригінального продукту.

					КПІ.IT-7311.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		24

2 МОДЕЛЮВАННЯ ТА КОНСТРУЮВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

2.1 Моделювання та аналіз програмного забезпечення

Моделювання програмного забезпечення доцільно розпочати з побудови схеми бізнес-процесів. Це дозволить прослідкувати весь шлях дій, що відбуваються в системі від запиту до відповіді. Особливо це знадобиться при конструюванні частин системи з великим ланцюгом подій. Таким буде, до прикладу, додавання нового замовника адміністратором:

- новий замовник переглядає список автомобілів та обирає той, що йому до вподоби;
- замовник повідомляє адміністраторові про намір оформити замовлення;
- адміністратор повідомляє йому про відсоток передплати;
- якщо замовник не погоджується – процес закінчується;
- якщо замовник погоджується, адміністратор запитує його дані та вносить їх у форму створення нового замовника в БД;
- у випадку, коли дані некоректні, адміністратор перепитує;
- якщо дані коректні, створюється запис у БД адміністративного сервіса;
- адміністративний сервіс відправляє повідомлення через RabbitMQ до основного сервісу про створення нового замовника;
- основний сервіс додає аналогічного замовника до власної БД;
- якщо передплата 0%, процес завершується;
- якщо передплата більше 0%, адміністратор очікує кошти від замовника;
- у випадку сплати процес завершується;

					КПІ.IT-7311.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		25

- у разі несплати адміністратор видаляє замовника з БД, адміністративний сервіс відправляє повідомлення про це основному сервісу, а той, в свою чергу, видаляє користувача у власній БД;

Описаний бізнес-процес зображено у вигляді BPMN-діаграми на рисунку 2.1.

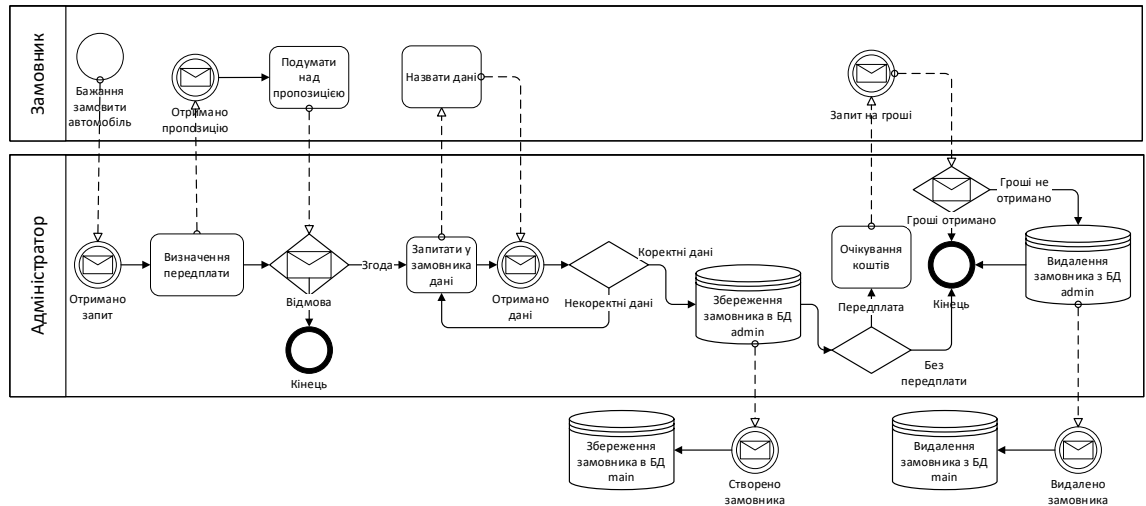


Рисунок 2.1 – BPMN-діаграма бізнес-процесу замовлення автомобіля

2.2 Проектування бази даних

В якості наступного етапу моделювання розроблюваного застосунку було обрано погляд на базу даних. Це базовий елемент кінцевої системи. Для вебзастосунку було спроектовано реляційну базу даних. Особливістю цього типу БД є нормалізація відношень у ній.

2.2.1 Аналіз предметної області

Після розгляду існуючих рішень в першому розділі, було отримано значне уявлення про автосалони. Проте, для отримання вичерпної інформації, необхідно провести подальше дослідження.

Автосалон - це магазин, що здійснює продаж нових або таких, що були у вжитку, автомобілів. Там можна обрати та купити автівку самостійно або ж за допомогою співробітників салону. Якщо бажане авто є в наявності, то можна одразу ж оформити його купівлю та забрати з салону. Також є

можливість оформити замовлення на автомобіль, якщо його нема в наявності, і забрати, як тільки воно прибуде в автосалон [4].

Необхідно виділити дані, якими оперують в автосалоні. З огляду на отриману інформацію можна зробити висновок, що це:

- співробітники підприємства: їхні особисті дані, адреса проживання, контактна інформація;
- посади співробітників: вимоги, відповідальність, зарплатня;
- автомобілі: виробник, модель, тип кузова, дата виготовлення, номер двигуна, опис, додаткове обладнання;
- замовлення: особиста інформація про замовника, дата замовлення, дата продажу, автомобіль, контактні дані і інше;
- виробник: назва, країна походження, адреса, опис.

Отже, після узагальнення інформації щодо предметної області, я перейшов до створення її моделі найвищого рівня абстракції, тобто інфологічної моделі. На її основі згодом можна буде створити схему за конкретною моделлю даних. В даному випадку обрано реляційну модель БД.

2.2.2 Побудова моделі предметної області

Для зручності задавання сутностей та зв'язків між ними при розробці вебзастосунку необхідно привести предметну область до конкретного виду. Для цього було обрано ER-модель.

2.2.2.1 Виділення сутностей

Внаслідок проведення аналізу, було виділено базові сутності цієї предметної області.

- Співробітник. Атрибути співробітника: ПІБ, вік, стать, адреса, номер телефону, паспортні дані, посада. Посада повинна бути окремою сутністю.

- Автомобіль. Атрибути автомобіля: модель, виробник, типу кузова, дата виробництва, колір, номер кузова, номер двигуна, опис, додаткове обладнання, ціна, відповідальний співробітник. Виробник, тип кузова, додаткове обладнання та співробітник – окремі сутності.
- Замовник. Атрибути замовника: ПІБ, адреса, номер телефону, паспортні дані, автомобіль, дата замовлення, дата продажу, відмітка про виконання, відмітка про оплату, відсоток передоплати, відповідальний співробітник. Автомобіль та співробітник – окремі сутності.

Базові сутності із атрибутами наведено у структурованому вигляді в таблиці 2.1.

Таблиця 2.1 – Базові сутності

Сутність	Атрибути
Співробітник	ПІБ; Вік; Стать; Адреса; Номер телефону; Паспортні дані; Посада
Автомобіль	Модель; Виробник; Тип кузова; Дата виробництва; Колір; Номер кузова; Номер двигуна; Опис; Додаткове обладнання; Ціна; Відповідальний співробітник
Замовник	ПІБ; Адреса; Номер телефону; Паспортні дані; Автомобіль; Дата замовлення; Дата продажу; Відмітка про виконання; Відмітка про оплату; Відсоток передоплати; Відповідальний співробітник

Для певних основних сутностей необхідно виділити також додаткові сутності. Їх наведено в таблиці 2.2.

Таблиця 2.2 – Додаткові сутності

Сутність	Атрибути
Посада	Найменування; Оклад; Обов'язки; Вимоги
Виробник	Найменування; Країна; Адреса; Опис; Відповідальний співробітник
Додаткове обладнання	Найменування; Характеристики; Ціна
Тип кузова	Найменування; Опис

На даному етапі вказані атрибути приведено без урахування нормалізації та формалізації. Це буде здійснено в наступних розділах.

2.2.2.2 Побудова ER-діаграми

Після того, як було виділено сутності та зв'язки, можна переходити до побудови інфологічної моделі у вигляді ER-діаграми. Було вирішено застосувати для цього нотацію Чена [5]. Також було застосовано розширений список позначень для різних типів сутностей [6]:

- стрижнева сутність – така сутність, що є незалежною від усіх інших;
- асоціативна сутність – виражає зв'язок «багато до багатьох», що виникає між двома сутностями. Наприклад, у студента може бути багато викладачів, а у викладача – багато студентів;
- характеристична сутність – слабка сутність, пов'язана із більш сильною зв'язком «один до багатьох» або «один до одного». Вона описує чи уточнює іншу сутність і при цьому повністю залежить від неї. Наприклад, сутність «Відпустка» є характеристикою для сутності «Співробітник». Із видаленням співробітника видаляються і всі його відпустки;
- сутність позначення – позначає іншу сутність, але при цьому є самостійною. Для прикладу, сутність «Вид» є позначенням для

сутності «Тварина». При цьому наявність запису конкретного виду все ще має сенс при зникненні тварин, що належать до цього виду. Умовні позначення діаграми за обраною нотацією наведено на рисунку 2.2.

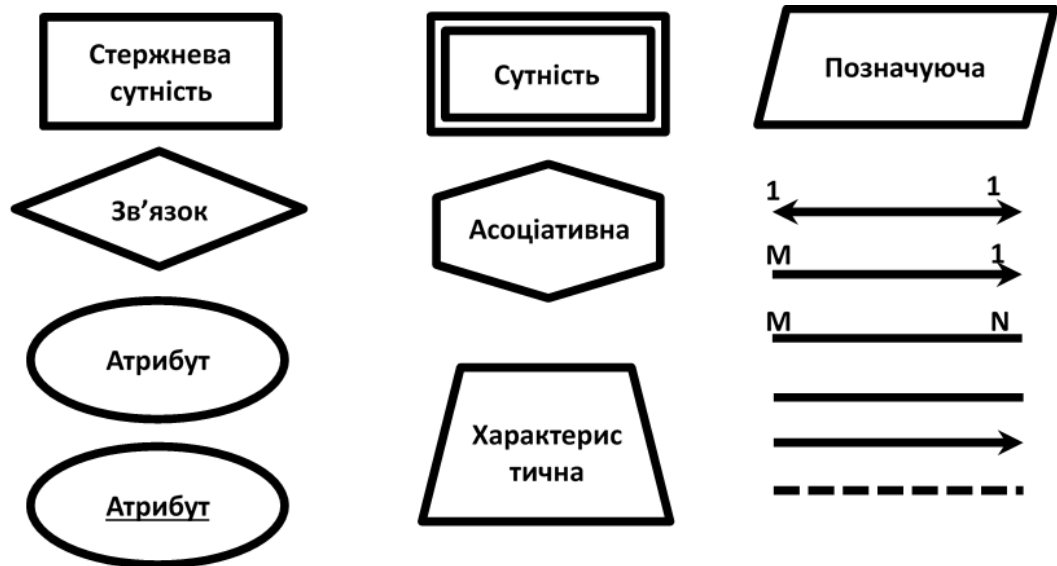


Рисунок 2.2 – Позначення ER-діаграми

Побудовану ER-діаграму за обраною нотацією наведено в графічному матеріалі КПІ.ІТ-7311.045440.06.99СБД.

2.2.3 Даталогічне проектування

На даному етапі передбачалося, що застосовуватиметься СУБД MySQL. Тому даталогічне проектування проводилося з урахуванням відповідних особливостей обраної системи. В той же час, створення таблиць конкретної БД буде здійснюватися засобами обраного вебфреймворку. Таким чином, передбачено те, що в майбутньому СУБД можна змінити за бажанням.

2.2.3.1 Формалізація зв'язків

Єдиний зв'язок «багато до багатьох», який необхідно формалізувати - це зв'язок між сутністю «Автомобіль» та «Додаткове обладнання». Ми допускаємо, що в кожному автомобілі може бути декілька додаткових обладнань. В той же час, кожен тип обладнання може бути встановлений в

декілька різних автомобілів. В такому випадку раціональним рішенням буде створення нової характеристичної сутності із трьома атрибутами: ID, ID автомобіля, ID обладнання. Проте, в ході проектування програмного продукту, було прийнято рішення про те, що ефективніше буде додати в сутність «Автомобіль» три атрибути: «ID додаткового обладнання 1», «ID додаткового обладнання 2» та «ID додаткового обладнання 3». Кожне з них може бути null. Це значить, що в одному автомобілі передбачається перелічення не більше, ніж трьох додаткових обладнань.

2.2.3.2 Введення обмежень цілісності

Всім сутностям бази даних необхідно встановити первинні ключі. Первинними ключами кожної сутності слугуватимуть ID. Вони будуть типу int із автоматичним заповненням. Жоден кортеж не матиме однакового значення атрибуту ID в межах однієї таблиці.

Стать співробітника позначається однією буквою. Номер телефону складається з цифр та знаків тире. Жодне з числових значень не може бути від'ємним. Значеннями атрибутів «Відмітка про оплату» та «Відмітка про виконання» можуть мати значення 0 або 1, що відповідає False та True. Значення атрибуту «Відсоток передоплати» від 0 до 100.

2.2.3.3 Нормалізація бази даних

Для всіх сутностей встановлено атрибут «ID», що є первинним ключем. Він встановлюється автоматично та його значення є унікальним у межах таблиці. Деякі атрибути передаються як зовнішні ключі. Виділено сутності-позначення: «Посада», «Виробник», «Тип кузова», «Додаткове обладнання». Для уникнення ситуації, коли на перетині стовпчика та рядка знаходиться множина значить, а не одне значення, в сутності «Автомобіль» є три атрибути: «Додаткове обладнання 1», «Додаткове обладнання 2», «Додаткове обладнання 3», кожне з яких містить одне значення.

Усі атрибути залежать лише від первинного ключа, тобто відсутні транзитивні функціональні залежності.

2.2.3.4 Визначення типів даних

Одержані типи даних для кожної сутності відображено в таблицях 2.3 – 2.9.

Таблиця 2.3 – Атрибути таблиці співробітників

Атрибут	Тип	Додаткова інформація
id	int (10)	PK
employee_adress	varchar (150)	not null
employee_age	int (10)	not null
employee_full_name	varchar (150)	not null
employee_passport_data	varchar (50)	not null
employee_phone	varchar (50)	not null
employee_sex	varchar (1)	not null
position_code_id	int (10)	FK, not null

Таблиця 2.4 – Атрибути таблиці автомобілів

Атрибут	Тип	Додаткова інформація
id	int (10)	PK
body_type_code_id	int (10)	FK, not null
car_body_type_number	int (10)	not null
car_color	varchar (50)	not null
car_date_of_production	date	not null
car_description	longtext	not null
car_engine_number	int (10)	not null
car_mark	varchar (50)	not null
car_price	int (10)	not null
employee_code_id	int (10)	FK, not null

Продовження таблиці 2.4

equipment_code_1_id	int (10)	FK
equipment_code_2_id	int (10)	FK
equipment_code_3_id	int (10)	FK

Таблиця 2.5 – Атрибути таблиці замовників

Атрибут	Тип	Додаткова інформація
id	int (10)	PK
car_code_id	int (10)	FK, not null
customer_adress	varchar (150)	not null
customer_data_of_sale	date	not null
customer_full_name	varchar (150)	not null
customer_mark_of_payment	tinyint (3)	not null
customer_mark_of_performance	tinyint (3)	not null
customer_order_data	date	not null
customer_passport_data	varchar (50)	not null
customer_phone	varchar (50)	not null
customer_prepayment_percentage	int (10)	not null
employee_code_id	int (10)	FK, not null

Таблиця 2.6 – Атрибути таблиці посад

Атрибут	Тип	Додаткова інформація
id	int (10)	PK
position_duties	varchar (200)	not null
position_name	varchar (50)	not null
position_requirements	varchar (200)	not null
position_salary	int (10)	not null

Таблиця 2.7 – Атрибути таблиці виробників

Атрибут	Тип	Додаткова інформація
id	int (10)	PK
employee_code_id	int (10)	FK, not null
maker_adress	varchar (150)	not null
maker_country	varchar (50)	not null
maker_description	longtext	not null
maker_name	varchar (50)	not null

Таблиця 2.8 – Атрибути таблиці додаткового обладнання

Атрибут	Тип	Додаткова інформація
id	int (10)	PK
equipment_characteristics	longtext	not null
equipment_name	varchar (50)	not null
equipment_price	int (10)	not null

Таблиця 2.9 – Атрибути таблиці типів кузовів

Атрибут	Тип	Додаткова інформація
id	int (10)	PK
body_type_description	longtext	not null
body_type_name	varchar (50)	not null

2.3 Вибір стеку технологій

Для наочності взаємодії мікросервісів, було вирішено розробляти елементи з використанням різних технологій. В той же час, свідомо було обмежено складність вивчення відповідних засобів розробки.

Для реалізації бекенду застосунку було обрано Python-фреймворки Django і Flask. В якості СУБД використано MySQL. Для написання

програмного коду застосовано інтегроване середовище розробки PyCharm, а також текстовий редактор notepad++. Фронтенд проєкту реалізовано з використанням JavaScript-бібліотеки React.

З метою перевірки роботи API застосовано програмний продукт Postman.

Після того, як стек технологій було обрано, розпочато ознайомлення з відповідними засобами та вивчення.

Python – високорівнева мова програмування, що на сьогодні має активну підтримку з боку її авторів та користується великою популярністю серед розробників. Вона підтримує декілька парадигм програмування: об'єктно-орієнтовану, аспектно-орієнтовану, процедурну, функціональну. Завдяки підтримці модулів та пакетів, а також зручній семантиці, Python чудово підходить для пришвидшеної розробки програмних продуктів різної складності та призначення. Наявність значної кількості фреймворків та бібліотек ще більше розширює область застосування цієї мови програмування.

В якості основного засобу було обрано Django. Це один з найпопулярніших фреймворків, що дозволяє створювати як бекенд, так і фронтенд вебзастосунку. В ньому значно спрощено багато речей, які забирають час при класичній розробці цього роду продуктів. Однією з вагомих переваг є те, що він надає дуже зручну панель адміністратора, для якої потрібно здійснювати мінімум налаштувань.

З дослідницькою метою було вирішено створювати застосунок на основі архітектури мікросервісів. Термін «мікросервіси» з'явився в середині минулого десятиліття, задля опису певного архітектурного стилю розробки програмних продуктів. Він став розповсюдженим завдяки розвитку практик гнучкої розробки, а також DevOps. Наразі йому приділяється багато уваги, присвячуються статті, дискусії, презентації на конференціях. Він активно розвивається та в цілому можна сказати, що це сучасний перспективний напрям.

					КПІ.IT-7311.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		35

Стиль мікросервісів – це такий підхід, при якому єдиний застосунок будується як сукупність невеличких, самостійних, незалежних і не тісно пов’язаних між собою сервісів, які спілкуються одне з одним з допомогою механізмів на кшталт HTTP, gRPC, AMQP. Ці сервіси побудовані довкола бізнес-потреб, тобто кожен відповідає за конкретний процес і розгортаються незалежно з використанням цілком автоматизованого середовища. Централізоване управління цими сервісами зведене до мінімуму. Самі вони можуть бути написані з використанням різних мов програмування та застосовувати різні технології зберігання даних.

Flask – як і Django, це фреймворк Python для розробки вебзастосунків. Проте, він є так званим мікрофреймворком, тобто сам по собі представляє ядро з мінімальною функціональністю, але підтримує нарощування за допомогою розширень. Я його обрав, виходячи з бажання випробувати взаємодію різнорідних мікросервісів через API. В той же час, його опанування не вимагало надто значних зусиль, тому що він базується на тій же мові програмування, що й Django.

Notepad++ - текстовий редактор з підтримкою підсвітки синтаксису та багатьох мов програмування. Він базується на популярному компоненті Scintilla, що написаний на C++ із застосуванням лише Windows API та STL, що визначає швидку роботу при мінімальному розмірі самої програми. Загалом, подібні продукти є базовими для активних користувачів комп’ютера, а особливо – розробників програмного забезпечення, тому що дозволяють переглядати і редагувати код практично будь-якої мови програмування чи розмітки.

PyCharm – інтегроване середовище розробки, призначене для мови програмування Python, що базується на IntelliJ IDEA. Наразі воно активно розвивається. Чудово підходить для розробки на Django.

В якості системи управління реляційними базами даних було обрано MySQL, проте обрані засоби розробки дозволяють абстрагуватися від

конкретної СУБД. Вона вільно розповсюджується та спершу створювалася як альтернатива комерційним аналогам. Широко підтримується різними мовами програмування та цілком відповідає потребам розроблюваного програмного продукту.

Загалом вебзастосунок пропонується розгортати на комп'ютерах в мережі автосалону, ізолювавши їх від доступу ззовні. Хоча, як альтернативу, можна розглядати і базування програмного засобу на віддалених серверах.

Попри те, що фронтенд застосунку можна реалізувати за допомогою Django, автором вирішено, що більш доцільним буде застосування окремого фреймворку. Як найбільш оптимальний варіант розглядається React.

React – це бібліотека Javascript для роботи із користувацькими інтерфейсами (UI). Її створили розробники Facebook. Вперше її почали використовувати на сайті цієї соціальної мережі в 2011 році. В 2013 Facebook відкрив вихідний код React.

За допомогою React розробники створюють вебзастосунки, що змінюють відображення без перезавантаження сторінки. Завдяки цьому застосунки швидко реагують на дії користувача: заповнення форм, додавання товарів до корзини, застосування фільтрів та таке інше.

React застосовується для відображення елементів користувацького інтерфейсу. Проте, ця бібліотека може повністю керувати фронтендом, якщо застосовувати її разом з бібліотеками для керування станом та роутингу. Наприклад, Redux та React Router.

Однією з основних особливостей React є універсальність. Цю бібліотеку можна використовувати на сервері та на мобільних платформах за допомогою React Native. Цей принцип називається Learn Once, Write Anywhere, тобто «навчися один раз та пиши де завгодно».

Також дуже важливою особливістю бібліотеки є декларативність. За допомогою React розробник описує, як компоненти інтерфейсу виглядають в

різних станах. Декларативний підхід значно скорочує код та робить його зрозумілим.

Мікросервіси реалізовано за допомогою Docker у вигляді контейнерів, що спілкуються через API. Перевірка працездатності здійснювалася за допомогою Postman.

Взаємодія між мікросервісами здійснюватиметься за допомогою брокера повідомлень RabbitMQ, що реалізує стандарт AMQP. Структурна схема компонентів показана в графічному матеріалі КП.ІТ-311.045440.06.99СС.

2.4 Конструювання програмного забезпечення

Вебзастосунок автосалону передбачається до застосування в службових цілях. Інформація, якою він оперуватиме, призначена для співробітників підприємства. Прикладом таких даних є інформація про персонал: контактні дані, посада, зарплатня, адреса проживання; інформація про автомобілі: виробник, рік випуску, комплектація, вартість; контракти з клієнтами на купівлю автомобіля: співробітник, що здійснив продаж, автомобіль, ім'я та контакти клієнта.

Передбачена видача інформації за запитами, із певними умовами пошуку в базі даних. Ось деякі з передбачених:

- список співробітників за посадою;
- список автомобілів за типом кузова;
- список автомобілів за виробником.

Для редагування збереженої в базі даних інформації передбачено адміністративну панель.

2.4.1 Адміністративний сервіс

Для початку необхідно встановити Django та Django REST Framework. Далі потрібно створити новий проєкт, під назвою admin. Після чого буде

					КП.ІТ-7311.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		38

встановлено вебсервер та утворено базові файли для роботи Django-застосунку, включно з базою даних СУБД SQLite.

Наступним кроком буде створення необхідних файлів для збирання образів та розгортання контейнерів Docker. Вони дозволяють автоматично виконувати команди завантаження необхідних засобів та запуску процесів. Всього три нових файли:

- Dockerfile. В ньому вписані команди, які користувач викликає для збірки образу. Docker виконає послідовно виконає їх, при застосуванні docker build.
- docker-compose.yml. Так як admin складається з кількох контейнерів, необхідно встановити зв'язки між ними. Тут вказуються залежності та конфігурація цих елементів. Команди для запуску процесів, порти, розташування бази даних.
- requirements.txt. Тут прописуються назви та версії пакетів, які будуть встановлені з допомогою «pip install» під час виконання команд в Dockerfile.

Вигляд відповідних документів наведено на рисунку 2.3.

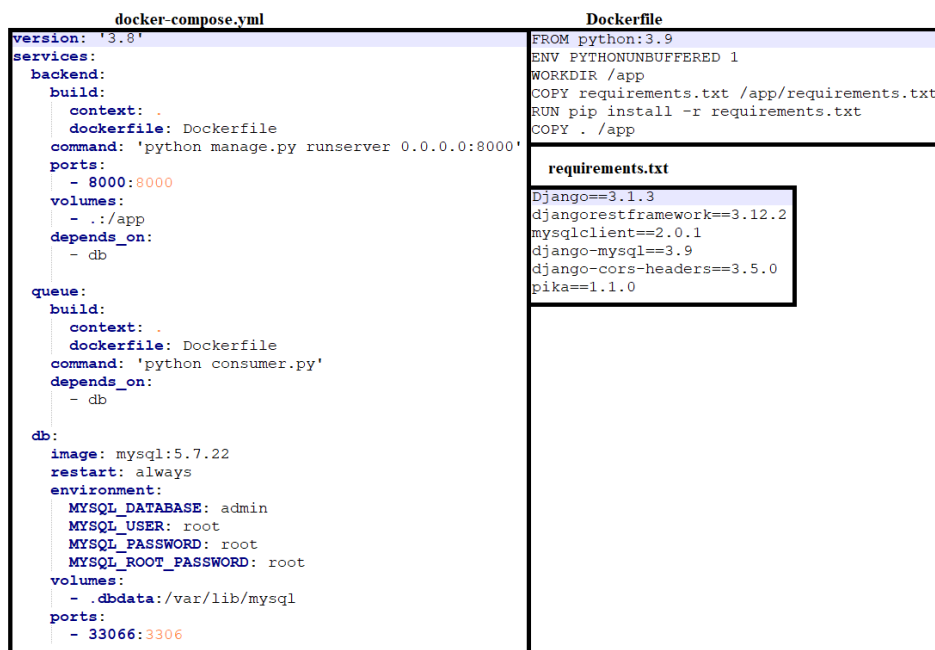


Рисунок 2.3 – Вміст файлів для створення Docker-контейнерів сервісу «admin»

Django дозволяє з самого старту запускати вебсервер для розробки. Для цього необхідно виконати команду «python3 manage.py runserver». Як тільки вебсервер запущено, можна перевірити його працездатність за URL <http://127.0.0.1:8000/>. Після перевірки зупиняємо його.

Далі потрібно заповнити «Dockerfile» та «requirements.txt», а також вписати в «docker-compose.yml» сервіс «backend». Для збірки образу та запуску контейнера виконується команда «docker-compose up». Застосунок Django доступний за тією ж адресою.

При створенні нового проекту фреймворк автоматично додає БД під управлінням СУБД SQLite, що є доволі зручно для цілей розробки. Проте, оскільки було обрано MySQL, потрібно виконати наступні дії:

- видалити файл «db.sqlite3»;
- додати до файлу «docker-compose.yml» сервіс «db» та відповідну залежність у сервісі «backend».

Після зупинки запущеного контейнеру знову введено команду «docker-compose up». Для перегляду БД встановлено плагін «DB Browser» для PyCharm. Після встановлення з'єднання підтверджено, що сервіс «db» працює коректно. У новоствореній базі даних на цей момент немає таблиць.

Щоб створити Django-застосунок автосалону, здійснено перехід до консолі в контейнері «backend» командою «docker-compose exec backend sh». Далі введено «python manage.py startapp carshowroom». Таким чином, у проєкті admin з'явилася нова директорія з файлами, вміст якої зображено на рисунку 2.4.

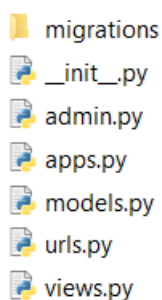


Рисунок 2.4 – Вміст директорії carshowroom

Після створення застосунку «carshowroom», до файлу settings.py необхідно внести зміни:

- додати рядки в секції «INSTALLED_APPS». Новий застосунок «carshowroom», а також «rest_framework» для створення REST API та «corsheaders» для доступу до ресурсів із інших доменів та портів;
- до «MIDDLEWARE» вписати рядок «corsheaders.middleware.CorsMiddleware». Також у файлі потрібно вписати «CORS_ORIGIN_ALLOW_ALL = True»;
- в «DATABASES». Структуру та вміст зображено на рисунку 2.5.

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.mysql',  
        'NAME': 'admin',  
        'USER': 'root',  
        'PASSWORD': 'root',  
        'HOST': 'db',  
        'PORT': '3306',  
        'default-character-set': 'utf8mb4',  
        'OPTIONS': {  
            'charset': 'utf8mb4'  
        }  
    }  
}
```

Рисунок 2.5 – Структура секції «DATABASES»

Наступний важливий крок – створення моделей сутностей предметної області в застосунку Django. Записуються вони у файлі models.py. Кожна з них є класом, що наслідує «django.db.models.Model». Відповідно до проведеного інфологічного та даталогічного моделювання було створено моделі сутностей (додаток А).

Деякі значення атрибутів мають обмеження. Для того, щоб їх ввести, підключено «django.core.validators». Також Django за замовчуванням надає зручний засіб для адміністрування. Було прийнято рішення використовувати цей інструмент в процесі розробки. З цією метою при описі атрибутів моделей також використовувався вибірковий аргумент «verbose name». Його

значенням є назва атрибуту, що відобразатиметься користувачеві адміністративної панелі Django.

Далі необхідно перейти в консоль контейнеру «backend». Для створення початкових міграцій потрібно виконати команди «python manage.py makemigrations» та «python manage.py migrate». Це дозволило не створювати БД на низькому рівні. Після завершення процедури можна скористатися DB Browser та переконатися в тому, що відповідна схема створена, що зображено на рисунку 2.6.

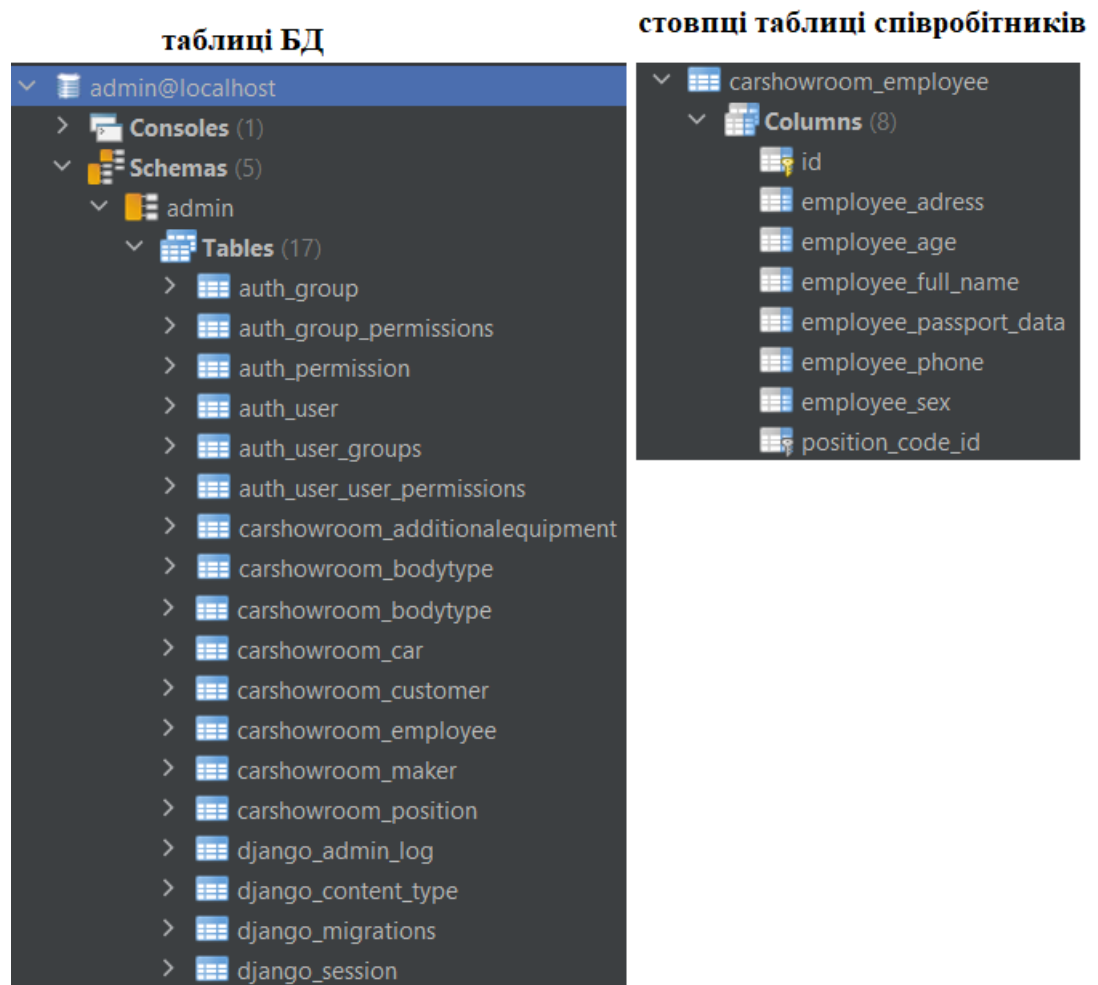


Рисунок 2.6 – Схема БД admin в DB Browser

Додано файл «serializers.py». Підключено «serializers» із «rest_framework». Імпортовано усі моделі із файлу «models.py». Після цього описано класи серіалізації, що наслідують «ModelSerializer». Таким чином забезпечено можливість переведення екземплярів моделей Django в прості

типи python і навпаки. Це, в свою чергу, дозволило перетворювати їх у JSON. Також з'явилася можливість десеріалізації.

У файлі «views.py» необхідно було описати функції, що прийматимуть вебзапити та надаватимуть вебвідповіді. Набір view-функцій у наборах для кожної моделі є однаковим та описаний у таблиці 2.10.

Таблиця 2.10 – Набір view-функцій кожної моделі

Функція	Запит та дані	Відповідь
list	отримання усіх об'єктів	повний набір об'єктів
create	створення нового об'єкта, дані – його атрибути	новий об'єкт, статус «HTTP_201_CREATED»
retrieve	отримання одного об'єкта, дані – його id	об'єкт з відповідним id або нічого, якщо такого не існує
update	зміна атрибутів об'єкта, дані – його id та нові значення атрибутів	оновлений об'єкт та статус «HTTP_202_ACCEPTED» або exception, якщо введені значення атрибутів не проходять валідацію
destroy	знищення об'єкта, дані – його id	статус «HTTP_204_NO_CONTENT»

Для адресування в Django-сервісі необхідно створити шаблони URL у файлі «urls.py», прописавши конфігурацію API. Використано можливість підключити файл з конфігурацією. Для цього імпортовано «path» та «include» із «django.urls». Записано до переліку «urlpatterns» новий шлях, «path('api/', include('carshowroom.urls'))». Створено відповідний файл у директорії «carshowroom». Встановлено зв'язок URL та view-функцій (додаток Б).

2.4.2 Основний сервіс

Для розробки основного сервісу обрано мікрофреймворк Flask. Створено у потрібній директорії файли:

- main.py. Тут прописані моделі, взаємодія з БД та API сервісу;
- Dockerfile;
- docker-compose.yml;
- requirements.txt;

В попередньому підрозділі описано, як заповнюються файли для створення образів Docker та запуск контейнерів. При цьому «Dockerfile» є аналогічним. Вміст «docker-compose.yml» та «requirements.txt» зображено на рисунку 2.7.

docker-compose.yml	requirements.txt
<pre>version: '3.8' services: backend: build: context: . dockerfile: Dockerfile command: 'python main.py' ports: - 8001:5000 volumes: - ../app depends_on: - db queue: build: context: . dockerfile: Dockerfile command: 'python consumer.py' depends_on: - db db: image: mysql:5.7.22 restart: always environment: MYSQL_DATABASE: main MYSQL_USER: root MYSQL_PASSWORD: root MYSQL_ROOT_PASSWORD: root volumes: - dbdata:/var/lib/mysql ports: - 33067:3306</pre>	<pre>Flask==1.1.2 Flask-SQLAlchemy==2.4.4 SQLAlchemy==1.3.20 Flask-Migrate==2.5.3 Flask-Script==2.0.6 Flask-Cors==3.0.9 requests==2.25.0 mysqlclient==2.0.1 pika==1.1.0</pre>

Рисунок 2.7 – Вміст файлів для створення Docker-контейнерів сервісу «main»

В «requirements.txt» вписано необхідні пакети. В «docker-compose.yml» додано сервіси «backend» та «db» з відповідної конфігурацією. Створюваний Flask-сервіс так само, як і Django-сервіс, використовує БД під управлінням СУБД MySQL.

Для створення образу та запуску контейнеру Docker, що коректно працюватиме, написано базовий застосунок Flask (рисунок 2.8).

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def index():
    return 'Hello'

if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0')
```

Рисунок 2.8 – Вміст файлу «main.py» базового Flask-застосунку

Наступним кроком потрібно виконати команду «python3 main.py». Після запуску вебсервера здійснено перехід за URL <http://0.0.0.0:5000/> та з'ясовано, що застосунок працює коректно. Після чого сервер зупинено.

Створено образи Docker командою «docker-compose up». Після завершення здійснено перехід за URL «http://localhost:8001» та підтверджено, що контейнер працює належним чином.

Далі підключено Flask із MySQL. За це відповідають наступні рядки: «from flask_sqlalchemy import SQLAlchemy», «app.config["SQLALCHEMY_DB_URI"]='mysql://root:root@db/main'», «db = SQLAlchemy(app)».

Схоже до Django, у Flask задаються моделі сутностей. Також, за допомогою SQLAlchemy, описуються таблиці БД та їх зв'язки. Це відображено в таблицях 2.11 – 2.17.

Таблиця 2.11 – Реалізація сутності Employee

Атрибут	Тип даних Python	Тип даних SQLAlchemy
id	int	Integer, primary_key=True, autoincrement=False
employee_full_name	str	String(50)
employee_age	int	Integer
employee_sex	str	String(1)
employee_adress	str	String(150)
employee_phone	str	String(50)
employee_passport_data	str	String(50)
position_code	int	Integer, ForeignKey('position.id'), nullable=False

Таблиця 2.12 – Реалізація сутності Position

Атрибут	Тип даних Python	Тип даних SQLAlchemy
id	int	Integer, primary_key=True, autoincrement=False
position_name	str	String(50)
position_salary	int	Integer
position_duties	str	String(200)
position_requirements	str	String(200)

Таблиця 2.13 – Реалізація сутності Maker

Атрибут	Тип даних Python	Тип даних SQLAlchemy
id	int	Integer, primary_key=True, autoincrement=False
maker_name	str	String(50)
maker_country	str	String(50)

Продовження таблиці 2.13

maker_adress	str	String(150)
maker_description	str	Text
employee_code	int	Integer, ForeignKey('employee.id'), nullable=False

Таблиця 2.14 – Реалізація сутності AdditionalEquipment

Атрибут	Тип даних Python	Тип даних SQLAlchemy
id	int	Integer, primary_key=True, autoincrement=False
equipment_name	str	String(50)
equipment_characteristics	str	Text
equipment_price	int	Integer

Таблиця 2.15 – Реалізація сутності BodyType

Атрибут	Тип даних Python	Тип даних SQLAlchemy
id	int	Integer, primary_key=True, autoincrement=False
body_type_name	str	String(50)
body_type_description	str	Text

Таблиця 2.16 – Реалізація сутності Car

Атрибут	Тип даних Python	Тип даних SQLAlchemy
id	int	Integer, primary_key=True, autoincrement=False
car_mark	str	String(50)

Продовження таблиці 2.16

maker_code	int	Integer, ForeignKey('maker.id'), nullable=False)
body_type_code	int	Integer, ForeignKey('body_type.id'), nullable=False
car_date_of_production	str	Date
car_color	str	String(50)
car_body_type_number	int	Integer
car_engine_number	int	Integer
car_description	str	Text
equipment_code_1	int	Integer
equipment_code_2	int	Integer
equipment_code_3	int	Integer
car_price	int	Integer
employee_code	int	Integer, ForeignKey('employee.id'), nullable=False

Таблиця 2.17 – Реалізація сутності Customer

Атрибут	Тип даних Python	Тип даних SQLAlchemy
id	int	Integer, primary_key=True, autoincrement=False
customer_full_name	str	String(50)
customer_adress	str	String(150)
customer_phone	str	String(50)
customer_passport_data	str	String(200)

Продовження таблиці 2.17

car_code	int	Integer, ForeignKey('car.id'), nullable=False
customer_order_data	str	Date
customer_data_of_sale	str	Date
customer_mark_of_performance	bool	Boolean
customer_mark_of_payment	bool	Boolean
customer_prepayment_percentage	int	Integer
employee_code	int	Integer, ForeignKey('employee.id'), nullable=False

Також, для встановлення зв'язків «один до багатьох», та «один до одного» потрібно записати відповідні рядки для тих сутностей, на які посилаються зовнішні ключі (рисунок 2.9).

```

Employee
makers = db.relationship('Maker', backref='employee', lazy=True)
cars = db.relationship('Car', backref='employee', lazy=True)
customers = db.relationship('Customer', backref='employee', lazy=True)

Position
employees = db.relationship('Employee', backref='position', lazy=True)

Maker
cars = db.relationship('Car', backref='maker', lazy=True)

BodyType
cars = db.relationship('Car', backref='bodyType', lazy=True)

Car
customer = db.relationship("Customer", uselist=False, backref="car")

```

Рисунок 2.9 – Зв'язки «один до багатьох» та «один до одного»
За створення міграцій відповідає розширення «flask_migrate». Створено файл «manager.py» з наступним наповненням:

```

from main import app, db

from flask_migrate import Migrate, MigrateCommand

from flask_script import Manager

```

```
migrate = Migrate(app, db)
```

```
manager = Manager(app)
```

```
manager.add_command('db', MigrateCommand)
```

```
if __name__ == '__main__':
```

```
    manager.run()
```

В запусненому контейнері Flask-сервісу використано консоль, що викликається командою «docker-compose exec backend sh». Після чого створено початкову міграцію БД. Це здійснено командами «python manager.py db init» та «python manager.py db migrate».

За допомогою плагіна «DB Browser» в PyCharm підключено до БД Flask-сервісу. Як результат, вдалося пересвідчитися в тому, що міграція спрацювала коректно. Оновлену схему БД можна побачити на рисунку 2.10.

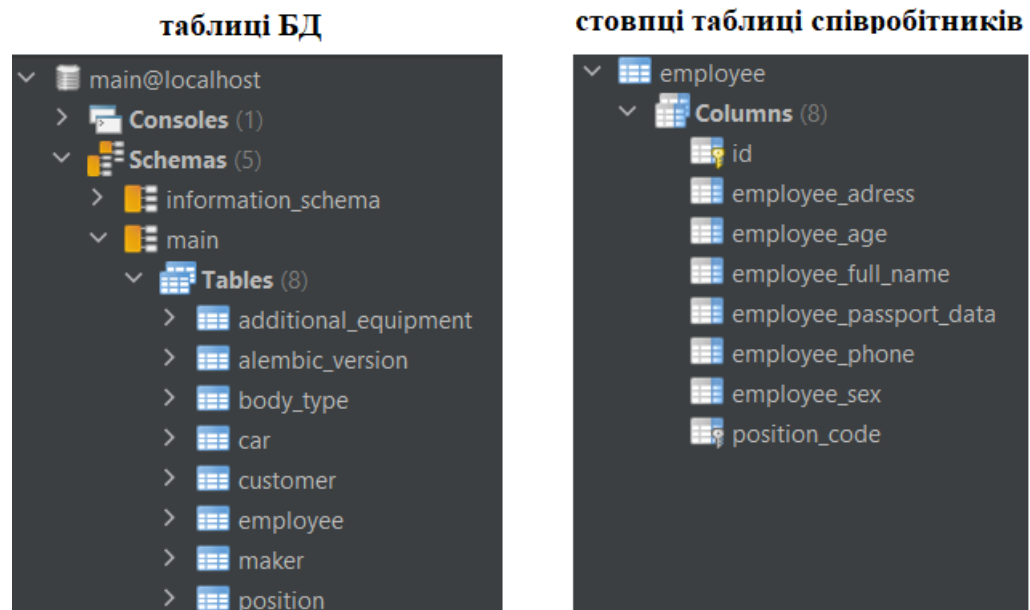


Рисунок 2.10 – Схема БД main в DB Browser

На відміну від Django, при створенні міграції в Flask до схеми БД додалися лише 8 сутностей. Тобто, ті, які були описані в якості моделей. Додаткові таблиці в БД «admin» - службові, що створює сам фреймворк Django.

Далі було прописано API. Оскільки сервіс «main» призначений лише для перегляду даних користувачем, а не зміни, достатньо було реалізувати тільки метод «GET». Шляхи записані за наступним принципом (на прикладі employees):

```
@app.route('/api/employees', methods=['GET'])
def all_employees():
    return jsonify(Employee.query.all())

@app.route('/api/employees/<int:id>', methods=['GET'])
def one_employee(id):
    return jsonify(Employee.query.get(id))
```

2.4.3 Брокер повідомлень

Попередньо було вирішено, що для обміну повідомленнями між компонентами «main» та «admin» буде використано стандарт AMQP. В якості конкретної реалізації обрано RabbitMQ, платформу з відкритим кодом. Вона є однією з найпопулярніших на сьогодні.

З метою спрощення розробки всієї системи було обрано хмарне рішення CloudAMQP. Це надійний сервіс, що пропонує багато локацій розміщення хмари та планів, в тому числі – безплатний. Такий підхід дозволяє зосередитися на розробці програмного продукту та послабити зв'язок компонентів системи.

Пройдено реєстрацію на сайті CloudAMQP. Обрано план «Little Lemur», що є безплатним. Локація – «amazon-web-services::eu-central-1», Франкфурт.

Для встановлення з'єднання з RabbitMQ в python є пакет «pika». Він був попередньо включений у файл «requirements.txt» як в сервісі «admin», так і «main» та на даному етапі вже присутній у образах Docker. Протестування повідомлень здійснюється з допомогою двох файлів:

- producer.py встановлює з'єднання із RabbitMQ та описує відправку повідомлень до черги іншого сервісу.

- consumer.py встановлює з'єднання із RabbitMQ, створює чергу та описує, як здійснюється прийом повідомлень із неї.

Відповідні файли було створено. Вміст «producer.py» сервісу «admin» виглядає наступним чином:

```
import pika
import json

params = pika.URLParameters('*URL хмарного RabbitMQ*')
connection = pika.BlockingConnection(params)
channel = connection.channel()

def publish(method, body):
    properties = pika.BasicProperties(method)
    channel.basic_publish(exchange="", routing_key='main',
body=json.dumps(body), properties=properties)
```

Код файлу «consumer.py» сервісу «main» приведено в додатку В. До файлу «docker-compose.yml» для обох контейнерів додано сервіс «queue». Після конструювання та запуску нового контейнеру в консолі отримано повідомлення «Started Consuming», що свідчить про успішну процедуру.

Сервіс «admin» надає інтерфейс для виконання усіх CRUD операцій з управління даними:

- create, створення;
- read, читання;
- update, оновлення;
- delete, видалення.

В той же час, сервіс «main» надає лише інтерфейс для «read», адже в ньому передбачений тільки перегляд даних. Необхідно синхронізувати дані, що зберігаються в БД обох компонентів. Для цього потрібно, щоб при виконанні будь-якої операції, окрім «read», сервіс «admin» відправляв повідомлення у чергу «main» із відповідними змінами. Після отримання

повідомлення від адміністративного, основний сервіс повторює зміну даних у власній БД. Для цього до функцій у «views.py» додано виклик «publish()». Аргументи наведено в таблиці 2.18.

Таблиця 2.18 – Аргументи функції «publish» у «views»

viewset	view	Аргументи
EmployeeViewSet	create	'employee_created', serializer.data
	update	'employee_updated', serializer.data
	destroy	'employee_deleted', pk
PositionViewSet	create	'position_created', serializer.data
	update	'position_updated', serializer.data
	destroy	'position_deleted', pk
MakerViewSet	create	'maker_created', serializer.data
	update	'maker_updated', serializer.data
	destroy	'maker_deleted', pk
AdditionalEquipmentViewSet	create	'additional_equipment_created', serializer.data
	update	'additional_equipment_updated', serializer.data
	destroy	'additional_equipment_deleted', pk
BodyTypeViewSet	create	'body_type_created', serializer.data
	update	'body_type_updated', serializer.data
	destroy	'body_type_deleted', pk
CarViewSet	create	'car_created', serializer.data
	update	'car_updated', serializer.data
	destroy	'car_deleted', pk
CustomerViewSet	create	'customer_created', serializer.data
	update	'customer_updated', serializer.data
	destroy	'customer_deleted', pk

Саме ці повідомлення обробляє consumer сервісу «main». Це забезпечує узгодженість даних у двох сервісах.

2.4.4 Клієнтський компонент

Для розробки фронтенду проєкту використано бібліотеку React.js. Автори надають обширну документацію українською мовою [7]. За їхніми ж ствердженнями, у нього є кілька визначних переваг:

- декларативний. React спрощує розробку інтерактивних користувацьких інтерфейсів. Достатньо лише описати те, як різні частини вашого інтерфейсу виглядають в кожному стані вебзастосунку і React ефективно оновить та відрендерить тільки потрібні компоненти, як тільки дані зміняться. Декларативні інтерфейси роблять код більш передбачуваним, при чому його набагато легше налагоджувати;
- заснований на компонентах. Створення інкапсульованих компонентів, які керують своїм власним станом, в свою чергу з них будується складний інтерфейс. Оскільки логіка компонентів написана на мові JavaScript, замість шаблонів, можна з легкістю передавати складні дані в застосунку та зберігати стан окремо від DOM;
- Вивчіть лише раз – пишіть будь-де. Автори не роблять припущень щодо стеку технологій, які використовуватиме розробник, тому можна розробляти нові функції в React, при цьому немає необхідності переписувати існуючий код.

Для створення нового проєкту на React, необхідно запустити команду «`npx create-react-app react-crud --template typescript`». Після завершення завантаження необхідних пакетів та збірки, запущено вебсервер командою «`npm start`». Перейшовши за URL <http://localhost:3000>, після виконання інструкцій, пересвідчено в тому, що «react-crud» працює належним чином. У

стартовому проєкті міститься зразковий код, який дозволяє краще зрозуміти, як описується інтерфейс та відбувається робота з даними в React.

В якості джерела стилів використано BootstrapCDN [8]. Для цього у файл «index.html» необхідно додати HTML-код, отриманий із сайту (версія 4.5.2):

```
<link rel="stylesheet" href=https://stackpath.bootstrapcdn.com/bootstrap/4.5.2/css/bootstrap.min.css  
integrity="sha384- JcKb8q3iqJ61gNV9KGb8thSsNjpSL0n8PARn9HuZOnI  
xN0hoP+VmmDGMN5t9UJ0Z" crossorigin="anonymous">
```

Компоненти «Nav» та «Menu» виділено окремо для повторного використання коду. Для цього створено директорію «admin/components» та файли «Menu.tsx» і «Nav.tsx» у ній. Написано код для створення вебсторінки із застосуванням обраних стилів:

- Nav.tsx навігаційна панель, розміщена вверху документа;
- Menu.tsx меню, розташоване в лівій частині документа. За ним здійснюється перехід між секціями адміністративної частини застосунку.

Обидва файли використано у «Wrapper.tsx», що, в свою чергу, слугує обгорткою. Кожна сторінка адміністративної частини використовуватиме її, а основний вміст сторінки буде розміщено всередині, у визначеному місці.

Створено файл «Employee.tsx» в директорії «admin». З цього моменту розпочинається взаємодія клієнта із API сервісів, що були попередньо зібрані як контейнери Docker. Створено папку «interfaces», в якій повинні зберігатися інтерфейси для даних, з якими працюватиме клієнт. Першим з них є «employee.ts»:

```
export interface Employee {  
  id: number;  
  employee_full_name: string;  
  employee_age: number;  
  employee_sex: string;
```

```

    employee_adress: string;
    employee_phone: string;
    employee_passport_data: string;
    position_code: number;
}

```

Щоб імпортувати його, потрібно вписати рядок «import {Employee} from "../interfaces/employee";». Для отримання всіх співробітників із БД адміністративного сервісу, необхідно використати fetch зі зверненням до API:

```
fetch('http://localhost:8000/api/employees')
```

Здійснюється це через використання хуку ефекта «useEffect». Отримані дані записуються у масив «employees» за допомогою «useState». Додано функцію видалення об'єкту. При її виконанні посилається запит на «http://localhost:8000/api/employees/\${id}» з методом DELETE.

В лівому верхньому кутку є кнопка, що містить посилання «/admin/employees/create». Об'єкти повинні відображатися у вигляді таблиці. Для цього було прописано заголовки та застосовано метод «map» до масиву «employees» і з кожного елемента записано значення атрибутів у новий рядок (таблиця 2.19)

Таблиця 2.19 – Відображення таблиці

Заголовок	Атрибут елемента
ПІБ	employee_full_name
Вік	employee_age
Стать	employee_sex
Адреса	employee_adress
Номер телефону	employee_phone
Паспортні дані	employee_passport_data
Посада	position_code

В кожному рядку є кнопка, яка посилає до «/admin/employees/{e.id}/edit», а також кнопка знищення елемента (відповідну функцію попередньо реалізовано).

Далі необхідно додати сторінки для створення нового елементу та редагування. Спершу «EmployeesCreate.tsx». Записано змінні для кожного атрибута об'єкта «Employee», їхні значення змінюватимуться за допомогою хука стану «useState». Реалізовано функцію надсилання «submit». Вона викликає fetch до «http://localhost:8000/api/employees» із методом POST. Значення атрибутів об'єкта записуються рядками в тіло JSON файлу. Записано форму, яка при надсиланні викликає функцію «submit». Містить поля «input», зміна значення в яких одразу призводить до зміни значення відповідного атрибута «Employee».

Файл «EmployeesEdit.tsx» має подібний вигляд, але при завантаженні виконується

«fetch('http://localhost:8000/api/employees/{props.match.params.id}')», Після чого поля форми заповнюються відповідними значеннями атрибутів отриманого об'єкта. При відправці форми здійснюється fetch до «fetch('http://localhost:8000/api/employees/{props.match.params.id}')» з методом PUT.

Створено файл «Main.tsx». Даний компонент працюватиме із API сервісу «main». Після його завантаження здійснюється «fetch('http://localhost:8001/api/employees')» для отримання усіх співробітників із БД. Вони записуються у масив, до якого застосовується метод «map». Для кожного елемента масиву створюється div, у якому записуються його атрибути.

Шляхи до компонентів записуються у файлі «App.tsx» наступним чином:

```
<BrowserRouter>
```

```
<Route path="/" exact component={Main}/>
```

```

    <Route path='/admin/employees' exact component={Employees}/>
    <Route path='/admin/employees/create' exact
component={EmployeesCreate}/>
    <Route path='/admin/employees/:id/edit' exact
component={EmployeesEdit}/>
  </BrowserRouter>

```

Інші компоненти записуються за аналогічним принципом. В результаті маємо клієнт, що працює з двома різними сервісами. Користувацький інтерфейс застосунку зображено в графічному матеріалі КПІ.IT-7311.045440.06.99КЕ.

2.5 Аналіз безпеки даних

В процесі розробки використовуються «debug» та «development» режими вебсерверів. Це зручно під час роботи над застосунком, але перед розгортанням програмного засобу в реальному середовищі для виконання прямих функцій, ці режими відключаються.

Логіни та паролі баз даних встановлюються відповідно до користувача, тому що. Якби вони були залишені «root», можна було би легко встановити з'єднання та вносити несанкціоновані зміни до БД.

Сервіси запаковано в Docker-контейнери. Порти захищені фаєрволом від доступу ззовні. Таким чином уникнуто можливості небажаного доступу ззовні.

Сервіси встановлюють з'єднання з RabbitMQ через захищений протокол amqp. Повідомлення, які вони відправляють, досягають свого адресата та не перехоплюються.

2.6 Висновки до розділу

В цьому розділі було проведено моделювання та конструювання програмного забезпечення. Це складалося з кількох етапів. В першу чергу –

створено схему структурну бізнес-процесів у вигляді діаграми BPMN проведено аналіз предметної області. На основі отриманих даних було здійснено інфологічне моделювання даних та узагальнено у вигляді ER-діаграми. За отриманою інфологічною моделлю проведено далалогічне моделювання з урахування особливостей СУБД MySQL.

Далі було спроектовано взаємодію компонентів вебзастосунку та зображено у відповідній структурній схемі. За отриманою структурною схемою здійснено проектування та розробку сервісів:

- створено компонент Django з БД;
- створено компонент Flask з БД;
- описано REST API та CRUD-операції обох сервісів;
- встановлено з'єднання з RabbitMQ та обмін повідомленнями;
- розгорнуто клієнтську частину застосунку з використанням React.

Також, зрештою, було проведено аналіз безпеки даних.

					КПІ.ІТ-7311.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		59

3 АНАЛІЗ ЯКОСТІ І ТЕСТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

3.1 Аналіз якості ПЗ

Якість програмного забезпечення визначається як спроможність виконувати запити замовника. В даному випадку в ролі замовника виступає сам розробник. Отже, для аналізу якості, спочатку необхідно окреслити, що від програмного забезпечення очікується. Для цього поставлено наступні умови успішності:

- запис, читання, оновлення та видалення даних з БД;
- опрацювання запитів до API;
- обмін повідомленнями між сервісами;
- узгодженість даних;
- взаємодія користувача з клієнтом.

Відповідно до зазначених вимог було визначено доцільні методи тестування.

3.2 Опис процесу тестування

Перевірку CRUD-операцій з БД можна провести як інтеграційний тест разом із тестуванням API. Недоліком є те, що якщо один із модулів працюватиме некоректно, буде важче виявити, який саме з них неробочий. Частково це питання вирішується при тестуванні Django-сервісу, адже цей фреймворк надає зручний засіб створення unit-тестів. Контроль виконання запису, оновлення та видалення даних в БД здійснюється через перегляд відповідних таблиць з допомогою плагіну «DB Browser» для PyCharm.

Тестування API здійснюється з допомогою Postman. Для цього прописується метод запиту, адреса, заголовки та тіло.

Сервіси обмінюються повідомленнями через RabbitMQ коли в «admin» здійснюється створення, зміна або видалення даних з БД. Для того, щоб

					КПІ.IT-7311.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		60

перевірити, чи отримує «main» повідомлення, до коду в файлі «consumer.py» додано команди виведення в консоль рядків на кшталт «Employee Created».

Перевірка узгодженості даних здійснюється шляхом перегляду таблиць БД після внесення змін до «admin» або виконанням GET-запиту до «main».

Взаємодія користувача з клієнтом перевіряється шляхом ручного тестування.

Тестування було виконано на персональному комп'ютері під управлінням операційної системи Windows 10. Обсяг оперативної пам'яті – 16 ГБ. Процесор – intel i5 7300HQ.

3.3 Контрольний приклад

Для кожного з зазначених критеріїв оцінки якості наведено контрольний приклад тесту. Детально процес тестування описано в документі «Програма та методика тестування».

3.3.1 Модульний тест

В даному тесті в БД створено об'єкт моделі «Position». Після чого новий запис отримано з БД та перевірено на відповідність вихідному об'єкту:

```
class PositionTestCase(TestCase):
```

```
    def setUp(self):
```

```
        Position.objects.create(position_name="Administrator",
                                position_salary=200000,
                                position_requirements="Syn vlasnyka avtosalonu",
                                position_duties="Upravlinnya avtosalonom")
```

```
    def test_position_fields_correct(self):
```

```
        administrator = Position.objects.get(position_name="Administrator")
        self.assertEqual(administrator.position_salary, 200000)
        self.assertEqual(administrator.position_requirements, "Syn vlasnyka avtosalonu")
```

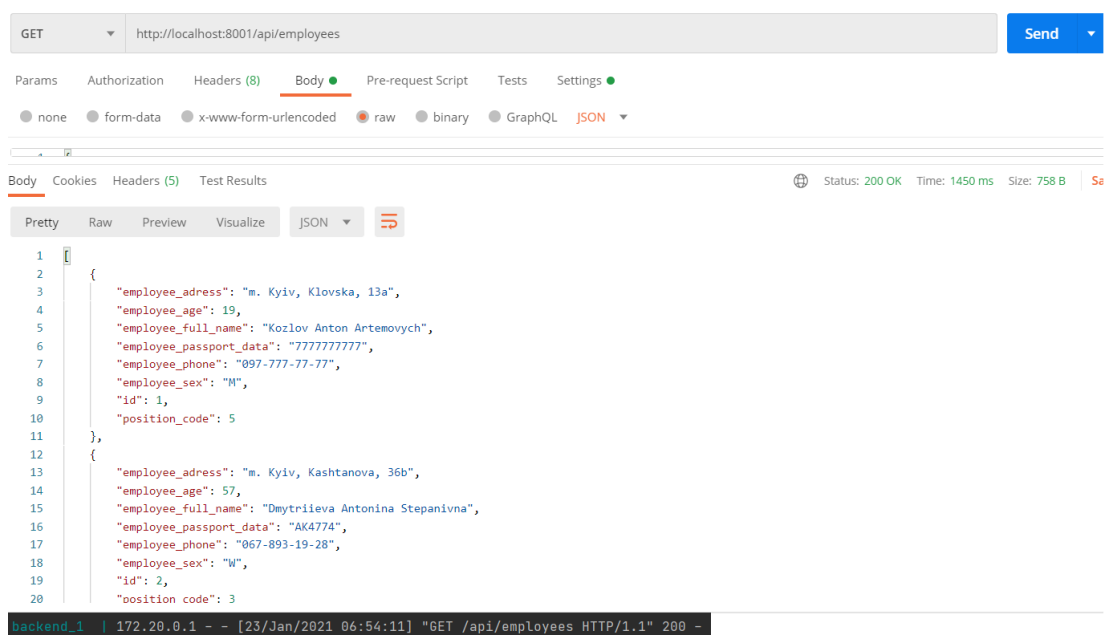
```
self.assertEqual(administrator.position_duties,  
avtosalonom")
```

"Upravlinnya

Результат успішний.

3.3.2 Тест API

На кінцеву точку API сервісу «admin», за адресою «<http://localhost:8000/api/employees>» відправлено GET-запит без тіла. У відповідь отримано JSON з переліком співробітників та статус-код 200, що значить успіх (рисуюнок 3.1).



Рисуюнок 3.1 – Результати тесту API

3.3.3 Тест повідомлень

На кінцеву точку API сервісу «admin», за адресою «<http://localhost:8000/api/employees/4>» відправлено PUT-запит з тілом, що містить об'єкт з новими значеннями атрибутів. У відповідь сервіс посилає статус 202 (прийнято) та відправляє повідомлення «employee_updated» до черги «main». Сервіс «main» отримує повідомлення, виконує аналогічну зміну елемента, виводить у консоль «Received in main», об'єкт з новими значеннями атрибутів та «Employee Updated». Засіб RabbitMQ Management також показує,

					КПІ.IT-7311.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		62

що повідомлення було відправлено та прийнято. Тестування зображене на рисунку 3.2.

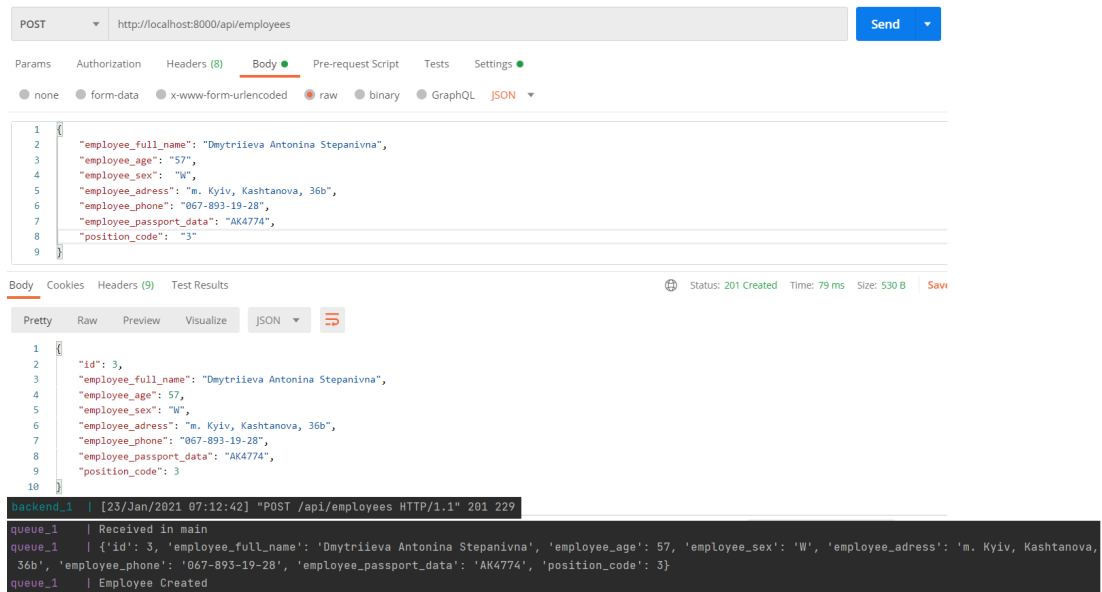


Рисунок 3.2 – Результат тесту повідомлень

3.3.4 Тест узгодженості даних

Після проведення минулого тесту запис у БД «main» повинен був змінитися. На кінцеві точки «`http://localhost:8000/api/employees/3`» та «`http://localhost:8001/api/employees/3`» відправлено GET-запити. У відповідь одержано відповіді зі статус-кодами 200 (успішно) та тілами, що містили об'єкти з ідентичними значеннями атрибутів.

3.3.5 Тест користувацького інтерфейсу

Відкрито адресу «`http://localhost:3000/admin/employees`». Виведено список усіх об'єктів «Employee» бази даних «main». На сторінці натиснуто на кнопку «Edit» першого запису таблиці. Здійснено перехід до форми редагування запису. Поля для вводу містять дані, що відповідають значенням атрибутів обраного об'єкта (рисунок 3.3).

Автосалон	Sign out
Employees	ПІБ
Positions	Kozlov Anton Artemovych
Makers	Вік
Additional Equipment	1
Body Types	Стать
Cars	М
Customers	Адреса
	m. Kyiv, Klovskya, 13a
	Номер телефону
	097-777-77-77
	Паспортні дані
	7777777777
	Посада
	4
	Зберегти

Рисунок 3.3 – Результат тесту користувацького інтерфейсу

3.4 Системні вимоги до ПЗ

Система, на якій розгортатиметься ПЗ, повинна бути під управлінням ОС Windows. Обумовлено це тим, що Docker найкраще працює саме на цій платформі. Найбільш актуальною версією на момент розробки застосунку є Windows 10.

На комп'ютер, перед розгортанням вебзастосунку, необхідно встановити наступні програмні засоби:

- Docker;
- npm;
- python.

3.5 Апаратні вимоги до ПЗ

Мінімальні апаратні вимоги до комп'ютера, на якому буде здійснено розгортання:

- процесор серії intel core i3 третього покоління або аналог;
- 8 ГБ оперативної пам'яті;
- 5 ГБ вільного дискового простору.

3.6 Висновки до розділу

В даному розділі було задано критерії аналізу якості ПЗ. Після постановки задач визначено й описано процес тестування. За описом складено контрольні тести, що дали успішний результат.

Визначено апаратні та системні вимоги до ПЗ. За їх дотримання розгорнутий застосунок повинен працювати коректно, з очікуваними результатами.

					КПІ.ІТ-7311.045440.02.81	Арк.
						65
Змін.	Арк.	№ докум.	Підп.	Дата.		

4 ВПРОВАДЖЕННЯ ТА СУПРОВІД ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

4.1 Розгортання програмного забезпечення

Розроблений вебзастосунок для управління автосалону складається із наступних компонентів:

- контейнер «admin» на Django, що відповідає за адміністративну частину програмного продукту;
- контейнер «main» на Flask, що відповідає за основну частину програмного продукту, призначену для перегляду даних;
- брокер повідомлень RabbitMQ;
- клієнт, написаний на React.

Необхідно встановити на комп'ютер засоби Docker Desktop, npm та python. Без них неможливо буде виконати команди розгортання контейнерів React-клієнта. Також необхідно зареєструвати план в Cloud AMQP. Після чого на сторінці керування скопіювати URL. Його потрібно вставити як аргумент в «pika.URLParameters('*місце для URL*')», що знаходиться у файлах «consumer.py» та «producer.py». Розгортати RabbitMQ не потрібно, він працює в хмарі.

4.1.1 Розгортання контейнера адміністративної частини

Кроки розгортання:

- перейти в директорію «admin» проєкту;
- відкрити в цьому місці консоль;
- ввести команду «docker-compose up».

Після виконання кроків необхідно почекати встановлення пакетів. Як результат, в Docker Desktop відобразяться нові образи та контейнери (рисунок 4.1).



Рисунок 4.1 – Сконструйовані та запущені процеси admin

4.1.2 Розгортання контейнера основної частини

Дії ідентичні до тих, що здійснені для «admin», за виключенням того, що потрібно перейти в директорію «main» проєкту. Результат виконання команди розгортання на рисунку 4.2.



Рисунок 4.2 – Сконструйовані та запущені процеси main

4.1.3 Розгортання клієнта

Кроки розгортання:

- перейти в директорію «front» та далі в «react-crud»;
- відкрити в цьому місці консоль;
- ввести команду «npm start»

4.2 Робота з програмним забезпеченням

Інструкція роботи користувача із клієнтом вебзастосунку для управління автосалоном наведена в документі «Керівництво користувача».

4.3 Висновки до розділу

В даному розділі було вказано послідовність дій для розгортання усіх компонентів вебзастосунку для управління автосалоном, розробленого в ході написання дипломного проєкту.

					КПІ.IT-7311.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		68

ВИСНОВКИ

В ході роботи над даним дипломним проєктом було розроблено вебзастосунок для управління автосалоном.

Застосовано визначений стек технологій. В якості середовища розробки обрано PyCharm. У якості СУБД взято MySQL. Для створенні бекенд-сервісів використано python-фреймворки Django та Flask. Фронтенд написаний з використанням бібліотеки React. Для обміну повідомленнями між бекенд-сервісами використано хмарне рішення RabbitMQ.

Було розглянуто наявні рішення та визначено аспекти функціональності вебзастосунку, на які варто звернути найбільшу увагу.

Після цього проведено детальний аналіз предметної області і виділено сутності. На основі отриманих даних здійснено інфологічне та даталогічне проектування, зображено схему бази даних.

Створено структурну схему компонентів програмного забезпечення. Результати проектування БД використано для опису моделей у Django та Flask сервісах. Реалізовано REST API. Налагоджено обмін повідомленнями за допомогою Cloud AMQP.

Створено клієнт вебзастосунку та налагоджено його зв'язок із сервісами «main» та «admin».

Результати роботи достатньо відповідають сучасному рівню наукових і технічних знань. Вагомим аргументом для цього є те, що для розробки програмного забезпечення було використано сучасні технології та доволі новий архітектурний стиль.

Результати роботи можна застосовувати за безпосереднім призначенням – управління автосалоном. Проте, більшу цінність вони мають як джерело вивчення принципів розробки мікросервісних застосунків.

Продовження досліджень за відповідною тематикою є доцільним, адже мікросервіси – перспективний напрямок. Можна дослідити, як функціонують більш складні за структурою мікросервісні застосунки, в яких сферах такий

підхід є придатним до використання та розглянути перспективи вдосконалення результатів роботи.

Було визначено критерії якості програмного забезпечення. Після чого визначено методи тестування та протестовано вебзастосунок.

Складено технічну документацію щодо розгортання модулів розробленої системи. Також написано інструкцію користувача.

Більшість поставлених в дипломному проектуванні завдань було цілком виконано. Однак, є аспекти, що варто доопрацювати:

- контроль введення інформації;
- обмеження доступу;
- захист від некоректних дій користувача;
- розширення та покращення функціоналу і користувацького інтерфейсу клієнта;
- оптимізація моделі даних;
- тестування на різних платформах.

ПЕРЕЛІК ВИКОРИСТАНИХ ДЖЕРЕЛ

- 1) Advantages and disadvantages of microservice architecture. URL: <https://cloudacademy.com/blog/microservices-architecture-challenge-advantage-drawback/> (дата звернення 23.05.2021).
- 2) Automotive Creatio. URL: <https://www.terrasoft.ru/industries/auto> (дата звертання 23.05.2021).
- 3) Альфа-Авто: Автосалон + Автосервіс + Автозапчастини. URL: https://rarus.com.ua/ru/catalog/avto/alfa_avto_avtosalon_avtoservis_avtozapchastini/ (дата звертання 23.05.2021).
- 4) Що таке Автосалон і як працює? Принцип роботи і для чого потрібен. URL: <https://auto.ria.com/uk/terms/avtosalon/> (дата звернення 19.04.2021).
- 5) ER-діаграми в нотації Чена. URL: https://studme.org/93800/informatika/er-diagramy_notatsii_chena (дата звернення 23.04.2021).
- 6) Класифікація сутностей. URL: <http://citforum.ru/database/dbguide/2-3.shtml> (дата звернення 16.04.2021).
- 7) Документація React.js. URL: <https://uk.reactjs.org/> (дата звернення 24.04.2021).
- 8) BootstrapCDN. URL: <https://www.bootstrapcdn.com/> (дата звернення 13.04.2021).

Додаток А

Лістинг моделей Django-сервісу

```
class Employee(models.Model):
    employee_full_name = models.CharField('ПІБ співробітника', max_length=50)
    employee_age = models.IntegerField('Вік співробітника')
    employee_sex = models.CharField('Стать', max_length=1)
    employee_adress = models.CharField('Адреса співробітника', max_length=150)
    employee_phone = models.SlugField('Телефон співробітника')
    employee_passport_data = models.CharField('Паспортні дані співробітника', max_length=200)
    position_code = models.ForeignKey('Position', on_delete=models.CASCADE, verbose_name='Посада')

    def __str__(self):
        return f'{self.position_code} | {self.employee_full_name}'

class Position(models.Model):
    position_name = models.CharField('Найменування посади', max_length=50)
    position_salary = models.IntegerField('Оклад')
    position_duties = models.CharField('Обов'язки', max_length=200)
    position_requirements = models.CharField('Вимоги', max_length=200)

    def __str__(self):
        return self.position_name

class Maker(models.Model):
    maker_name = models.CharField('Найменування виробника', max_length=50)
    maker_country = models.CharField('Країна', max_length=50)
    maker_adress = models.CharField('Адреса виробника', max_length=150)
    maker_description = models.TextField('Опис')
    employee_code = models.ForeignKey(Employee, on_delete=models.CASCADE,
    verbose_name='Співробітник')

    def __str__(self):
        return self.maker_name

class AdditionalEquipment(models.Model):
    equipment_name = models.CharField('Найменування обладнання', max_length=50)
    equipment_characteristics = models.TextField('Характеристики обладнання')
    equipment_price = models.IntegerField('Ціна обладнання')

    def __str__(self):
        return self.equipment_name

class BodyType(models.Model):
    body_type_name = models.CharField('Найменування кузова', max_length=50)
    body_type_description = models.TextField('Опис кузова')

    def __str__(self):
        return self.body_type_name

class Car(models.Model):
    car_mark = models.CharField('Марка', max_length=50)
    maker_code = models.ForeignKey(Maker, on_delete=models.CASCADE, verbose_name='Виробник')
    body_type_code = models.ForeignKey(BodyType, on_delete=models.CASCADE, verbose_name='Тип
кузова')
    car_date_of_production = models.DateField('Дата виробництва')
    car_color = models.CharField('Колір', max_length=50)
    car_body_type_number = models.IntegerField('Номер кузова')
    car_engine_number = models.IntegerField('Номер двигуна')
```

					КПІ.IT-7311.045440.02.81	Арк.
						72
Змін.	Арк.	№ докум.	Підп.	Дата.		


```

car_description = models.TextField('Опис автомобіля')
equipment_code_1 = models.ForeignKey(AdditionalEquipment, related_name='+',
                                     on_delete=models.CASCADE, blank=True, null=True,
verbose_name='Обладнання 1')
equipment_code_2 = models.ForeignKey(AdditionalEquipment, related_name='+',
                                     on_delete=models.CASCADE, blank=True, null=True,
verbose_name='Обладнання 2')
equipment_code_3 = models.ForeignKey(AdditionalEquipment, related_name='+',
                                     on_delete=models.CASCADE, blank=True, null=True,
verbose_name='Обладнання 3')
car_price = models.IntegerField('Ціна автомобіля')
employee_code = models.ForeignKey(Employee, on_delete=models.CASCADE,
verbose_name='Співробітник')

def __int__(self):
    return self.pk

class Customer(models.Model):
    customer_full_name = models.CharField('ПІБ замовника', max_length=50)
    customer_adress = models.CharField('Адреса замовника', max_length=150)
    customer_phone = models.SlugField('Телефон замовника')
    customer_passport_data = models.CharField('Паспортні дані замовника', max_length=200)
    car_code = models.ForeignKey(Car, on_delete=models.CASCADE, verbose_name='Автомобіль')
    customer_order_data = models.DateField('Дата замовлення')
    customer_data_of_sale = models.DateField('Дата продажу')
    customer_mark_of_performance = models.BooleanField('Відмітка про виконання')
    customer_mark_of_payment = models.BooleanField('Відмітка про оплату')
    customer_prepayment_percentage = models.IntegerField(
        default=0,
        validators=[
            MaxValueValidator(100),
            MinValueValidator(0)
        ]
    )
    employee_code = models.ForeignKey(Employee, on_delete=models.CASCADE,
verbose_name='Співробітник')

```

Додаток Б

Вміст файлу urls.py Django-сервісу

```
from django.urls import path

from .views import *

urlpatterns = [
    path('employees', EmployeeViewSet.as_view({
        'get': 'list',
        'post': 'create',
    })),
    path('employees/<str:pk>', EmployeeViewSet.as_view({
        'get': 'retrieve',
        'put': 'update',
        'delete': 'destroy'
    })),
    path('positions', PositionViewSet.as_view({
        'get': 'list',
        'post': 'create',
    })),
    path('positions/<str:pk>', PositionViewSet.as_view({
        'get': 'retrieve',
        'put': 'update',
        'delete': 'destroy'
    })),
    path('makers', MakerViewSet.as_view({
        'get': 'list',
        'post': 'create',
    })),
    path('makers/<str:pk>', MakerViewSet.as_view({
        'get': 'retrieve',
        'put': 'update',
        'delete': 'destroy'
    })),
    path('additional_equipments', AdditionalEquipmentViewSet.as_view({
        'get': 'list',
        'post': 'create',
    })),
    path('additional_equipments/<str:pk>', AdditionalEquipmentViewSet.as_view({
        'get': 'retrieve',
        'put': 'update',
        'delete': 'destroy'
    })),
    path('body_types', BodyTypeViewSet.as_view({
        'get': 'list',
        'post': 'create',
    })),
    path('body_types/<str:pk>', BodyTypeViewSet.as_view({
        'get': 'retrieve',
        'put': 'update',
        'delete': 'destroy'
    })),
    path('cars', CarViewSet.as_view({
        'get': 'list',
        'post': 'create',
    })),
    path('cars/<str:pk>', CarViewSet.as_view({
        'get': 'retrieve',
```

					КПІ.IT-7311.045440.02.81	Арк.
						74
Змін.	Арк.	№ докум.	Підп.	Дата.		

```

        'put': 'update',
        'delete': 'destroy'
    )),
    path('customers', CustomerViewSet.as_view({
        'get': 'list',
        'post': 'create',
    })),
    path('customers/<str:pk>', CustomerViewSet.as_view({
        'get': 'retrieve',
        'put': 'update',
        'delete': 'destroy'
    })))
]

```

Додаток В

Вміст файлу «consumer.py» сервісу «main»

```
import pika
import json

from main import *

params =
pika.URLParameters('amqps://guomtwqy:0_BmGYhmT09RXj3IcxXwpULhI0_KAZ4V@sparrow.rmcloudamqp.com/guomtwqy')

connection = pika.BlockingConnection(params)

channel = connection.channel()

channel.queue_declare(queue='main')

def callback(ch, method, properties, body):
    print('Received in main')
    data = json.loads(body)
    print(data)

    if properties.content_type == 'employee_created':
        employee = Employee(id=data['id'],
                             employee_full_name=data['employee_full_name'],
                             employee_age=data['employee_age'],
                             employee_sex=data['employee_sex'],
                             employee_adress=data['employee_adress'],
                             employee_phone=data['employee_phone'],
                             employee_passport_data=data['employee_passport_data'],
                             position_code=data['position_code'])
        db.session.add(employee)
        db.session.commit()
        print('Employee Created')

    elif properties.content_type == 'employee_updated':
        employee = Employee.query.get(data['id'])
        employee.employee_full_name = data['employee_full_name']
        employee.employee_age = data['employee_age']
        employee.employee_sex = data['employee_sex']
        employee.employee_adress = data['employee_adress']
        employee.employee_phone = data['employee_phone']
        employee.employee_passport_data = data['employee_passport_data']
        employee.position_code = data['position_code']
        db.session.commit()
        print('Employee Updated')

    elif properties.content_type == 'employee_deleted':
        employee = Employee.query.get(data)
        db.session.delete(employee)
        db.session.commit()
        print('Employee Deleted')

    elif properties.content_type == 'position_created':
        position = Position(id=data['id'],
                             position_name=data['position_name'],
                             position_salary=data['position_salary'],
                             position_duties=data['position_duties'],
```

					КПІ.IT-7311.045440.02.81	Арк.
						76
Змін.	Арк.	№ докум.	Підп.	Дата.		

```

        position_requirements=data['position_requirements'])
db.session.add(position)
db.session.commit()
print('Position Created')

elif properties.content_type == 'position_updated':
    position = Position.query.get(data['id'])
    position.position_name = data['position_name']
    position.position_salary = data['position_salary']
    position.position_duties = data['position_duties']
    position.position_requirements = data['position_requirements']
    db.session.commit()
    print('Position Updated')

elif properties.content_type == 'position_deleted':
    position = Position.query.get(data)
    db.session.delete(position)
    db.session.commit()
    print('Position Deleted')

elif properties.content_type == 'maker_created':
    maker = Maker(id=data['id'],
                  maker_name=data['maker_name'],
                  maker_country=data['maker_country'],
                  maker_adress=data['maker_adress'],
                  maker_description=data['maker_description'],
                  employee_code=data['employee_code'])
    db.session.add(maker)
    db.session.commit()
    print('Maker Created')

elif properties.content_type == 'maker_updated':
    maker = Maker.query.get(data['id'])
    maker.maker_name = data['maker_name']
    maker.maker_country = data['maker_country']
    maker.maker_adress = data['maker_adress']
    maker.maker_description = data['maker_description']
    maker.employee_code = data['employee_code']
    db.session.commit()
    print('Maker Updated')

elif properties.content_type == 'maker_deleted':
    maker = Maker.query.get(data)
    db.session.delete(maker)
    db.session.commit()
    print('Maker Deleted')

elif properties.content_type == 'additional_equipment_created':
    additional_equipment = AdditionalEquipment(id=data['id'],
                                                equipment_name=data['equipment_name'],
                                                equipment_price=data['equipment_price'],
                                                equipment_characteristics=data['equipment_characteristics'])
    db.session.add(additional_equipment)
    db.session.commit()
    print('AdditionalEquipment Created')

elif properties.content_type == 'additional_equipment_updated':
    additional_equipment = AdditionalEquipment.query.get(data['id'])
    additional_equipment.equipment_name = data['equipment_name']
    additional_equipment.equipment_price = data['equipment_price']
    additional_equipment.equipment_characteristics = data['equipment_characteristics']
    db.session.commit()

```

```

print('AdditionalEquipment Updated')

elif properties.content_type == 'additional_equipment_deleted':
    additional_equipment = AdditionalEquipment.query.get(data)
    db.session.delete(additional_equipment)
    db.session.commit()
    print('AdditionalEquipment Deleted')

elif properties.content_type == 'body_type_created':
    body_type = BodyType(id=data['id'],
                          body_type_name=data['body_type_name'],
                          body_type_description=data['body_type_description'])
    db.session.add(body_type)
    db.session.commit()
    print('BodyType Created')

elif properties.content_type == 'body_type_updated':
    body_type = AdditionalEquipment.query.get(data['id'])
    body_type.body_type_name = data['body_type_name']
    body_type.body_type_description = data['body_type_description']
    db.session.commit()
    print('BodyType Updated')

elif properties.content_type == 'body_type_deleted':
    body_type = AdditionalEquipment.query.get(data)
    db.session.delete(body_type)
    db.session.commit()
    print('BodyType Deleted')

elif properties.content_type == 'car_created':
    car = Car(id=data['id'],
              car_mark=data['car_mark'],
              maker_code=data['maker_code'],
              body_type_code=data['body_type_code'],
              car_date_of_production=data['car_date_of_production'],
              car_color=data['car_color'],
              car_body_type_number=data['car_body_type_number'],
              car_engine_number=data['car_engine_number'],
              car_description=data['car_description'],
              equipment_code_1=data['equipment_code_1'],
              equipment_code_2=data['equipment_code_2'],
              equipment_code_3=data['equipment_code_3'],
              car_price=data['car_price'],
              employee_code=data['employee_code'])
    db.session.add(car)
    db.session.commit()
    print('Car Created')

elif properties.content_type == 'car_updated':
    car = Car.query.get(data['id'])
    car.car_mark = data['car_mark']
    car.maker_code = data['maker_code']
    car.body_type_code = data['body_type_code']
    car.car_date_of_production = data['car_date_of_production']
    car.car_color = data['car_color']
    car.car_body_type_number = data['car_body_type_number']
    car.car_engine_number = data['car_engine_number']
    car.car_description = data['car_description']
    car.equipment_code_1 = data['equipment_code_1']
    car.equipment_code_2 = data['equipment_code_2']
    car.equipment_code_3 = data['equipment_code_3']
    car.car_price = data['car_price']

```

```

car.employee_code = data['employee_code']
db.session.commit()
print('Car Updated')

elif properties.content_type == 'car_deleted':
    car = Car.query.get(data)
    db.session.delete(car)
    db.session.commit()
    print('Car Deleted')

elif properties.content_type == 'customer_created':
    customer = Customer(id=data['id'],
                        customer_full_name=data['customer_full_name'],
                        customer_adress=data['customer_adress'],
                        customer_phone=data['customer_phone'],
                        customer_passport_data=data['customer_passport_data'],
                        car_code=data['car_code'],
                        customer_order_data=data['customer_order_data'],
                        customer_data_of_sale=data['customer_data_of_sale'],
                        customer_mark_of_performance=data['customer_mark_of_performance'],
                        customer_mark_of_payment=data['customer_mark_of_payment'],
                        customer_prepayment_percentage=data['customer_prepayment_percentage'],
                        employee_code=data['employee_code'],)
    db.session.add(customer)
    db.session.commit()
    print('Customer Created')

elif properties.content_type == 'customer_updated':
    customer = Customer.query.get(data['id'])
    customer.customer_full_name = data['customer_full_name']
    customer.customer_adress = data['customer_adress']
    customer.customer_phone = data['customer_phone']
    customer.customer_passport_data = data['customer_passport_data']
    customer.car_code = data['car_code']
    customer.customer_mark_of_performance = data['customer_mark_of_performance']
    customer.customer_mark_of_payment = data['customer_mark_of_payment']
    customer.customer_prepayment_percentage = data['customer_prepayment_percentage']
    customer.employee_code = data['employee_code']
    db.session.commit()
    print('Customer Updated')

elif properties.content_type == 'customer_deleted':
    customer = Customer.query.get(data)
    db.session.delete(customer)
    db.session.commit()
    print('Customer Deleted')
channel.basic_consume(queue='main', on_message_callback=callback, auto_ack=True)
print('Started Consuming')
channel.start_consuming()
channel.close()

```

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2022 р.

ВЕБЗАСТОСУНК ДЛЯ УПРАВЛІННЯ АВТОСАЛОНОМ

Опис програми

КП.ІТ-7311.045440.03.13

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Олена ХАЛУС

Нормоконтроль:

_____ Катерина ЛІЩУК

Виконавець:

_____ Олександр ПАНЬКІВ

Київ – 2022

ТЕКСТ ПРОГРАМНОГО КОДУ

Тексти програмного коду

Вебзастосунок для управління автосалоном

(Найменування програми (документа))

CD-R

(Вид носія даних)

15 арк, 227000 Кб

(Обсяг програми, арк., Кб)

Київ – 2022

					КПІ.IT-7311.045440.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		2

Файл manage.py:

```
#!/usr/bin/env python
"""Django's command-line utility for administrative tasks."""
import os
import sys

def main():
    """Run administrative tasks."""
    os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'admin.settings')
    try:
        from django.core.management import execute_from_command_line
    except ImportError as exc:
        raise ImportError(
            "Couldn't import Django. Are you sure it's installed and "
            "available on your PYTHONPATH environment variable? Did you "
            "forget to activate a virtual environment?"
        ) from exc
    execute_from_command_line(sys.argv)

if __name__ == '__main__':
    main()
```

Файл admin.py:

```
from django.contrib import admin

from .models import *

class PositionAdmin(admin.ModelAdmin):
    list_display = ('id', 'position_name', 'position_salary', 'position_duties', 'position_requirements')
    list_display_links = ('id', 'position_name')
    search_field = ('position_name')

class EmployeeAdmin(admin.ModelAdmin):
    list_display = ('id', 'employee_full_name', 'employee_age', 'employee_sex', 'employee_adress',
'employee_phone', 'employee_passport_data', 'position_code')
    list_display_links = ('id', 'employee_full_name')
    search_field = ('employee_full_name')
    list_filter = ('position_code',)

class MakerAdmin(admin.ModelAdmin):
    list_display = ('id', 'maker_name', 'maker_country', 'maker_adress', 'employee_code')
    list_display_links = ('id', 'maker_name')
    search_field = ('maker_name')

class AdditionalEquipmentAdmin(admin.ModelAdmin):
    list_display = ('id', 'equipment_name', 'equipment_price')
    list_display_links = ('id', 'equipment_name')
    search_field = ('equipment_name')

class BodyTypeAdmin(admin.ModelAdmin):
    list_display = ('id', 'body_type_name')
    list_display_links = ('id', 'body_type_name')
    search_field = ('body_type_name')

class CarAdmin(admin.ModelAdmin):
    list_display = ('id', 'car_mark', 'maker_code', 'body_type_code', 'car_date_of_production', 'car_color')
    list_display_links = ('id', 'car_mark')
    search_field = ('car_mark')
    list_filter = ('body_type_code', 'maker_code')

class CustomerAdmin(admin.ModelAdmin):
```

					КПІ.IT-7311.045440.03.13	Арк.
						3
Змін.	Арк.	№ докум.	Підп.	Дата.		

```
list_display = ('id', 'customer_full_name', 'customer_adress', 'customer_phone', 'car_code',
'customer_mark_of_performance', 'customer_mark_of_payment')
list_display_links = ('id', 'customer_full_name')
search_field = ('customer_full_name')
list_filter = ('customer_mark_of_performance', 'customer_mark_of_payment')
```

```
admin.site.register(Employee, EmployeeAdmin)
admin.site.register(Position, PositionAdmin)
admin.site.register(Maker, MakerAdmin)
admin.site.register(AdditionalEquipment, AdditionalEquipmentAdmin)
admin.site.register(BodyType, BodyTypeAdmin)
admin.site.register(Car, CarAdmin)
admin.site.register(Customer, CustomerAdmin)
# Register your models here.
```

Файл serializers.py:

```
from rest_framework import serializers
from models import *

class EmployeeSerializer(serializers.ModelSerializer):
    class Meta:
        model = Employee
        fields = '__all__'

class PositionSerializer(serializers.ModelSerializer):
    class Meta:
        model = Position
        fields = '__all__'

class MakerSerializer(serializers.ModelSerializer):
    class Meta:
        model = Maker
        fields = '__all__'

class AdditionalEquipmentSerializer(serializers.ModelSerializer):
    class Meta:
        model = AdditionalEquipment
        fields = '__all__'

class BodyTypeSerializer(serializers.ModelSerializer):
    class Meta:
        model = BodyType
        fields = '__all__'

class CarSerializer(serializers.ModelSerializer):
    class Meta:
        model = Car
        fields = '__all__'

class CustomerSerializer(serializers.ModelSerializer):
    class Meta:
        model = Customer
        fields = '__all__'
```

Файл admin/urls:

```
from django.contrib import admin
from django.urls import path, include
urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('carshowroom.urls')),
]
```

					КПІ.ІТ-7311.045440.03.13	Арк.
Вмін.	Арк.	№ докум.	Підп.	Дата.		4

Файл carshowroom/urls:

```
from django.urls import path
```

```
from .views import *
```

```
urlpatterns = [  
    path('employees', EmployeeViewSet.as_view({  
        'get': 'list',  
        'post': 'create',  
    })),  
    path('employees/<str:pk>', EmployeeViewSet.as_view({  
        'get': 'retrieve',  
        'put': 'update',  
        'delete': 'destroy'  
    })),  
    path('positions', PositionViewSet.as_view({  
        'get': 'list',  
        'post': 'create',  
    })),  
    path('positions/<str:pk>', PositionViewSet.as_view({  
        'get': 'retrieve',  
        'put': 'update',  
        'delete': 'destroy'  
    })),  
    path('makers', MakerViewSet.as_view({  
        'get': 'list',  
        'post': 'create',  
    })),  
    path('makers/<str:pk>', MakerViewSet.as_view({  
        'get': 'retrieve',  
        'put': 'update',  
        'delete': 'destroy'  
    })),  
    path('additional equipments', AdditionalEquipmentViewSet.as_view({  
        'get': 'list',  
        'post': 'create',  
    })),  
    path('additional equipments/<str:pk>', AdditionalEquipmentViewSet.as_view({  
        'get': 'retrieve',  
        'put': 'update',  
        'delete': 'destroy'  
    })),  
    path('body_types', BodyTypeViewSet.as_view({  
        'get': 'list',  
        'post': 'create',  
    })),  
    path('body_types/<str:pk>', BodyTypeViewSet.as_view({  
        'get': 'retrieve',  
        'put': 'update',  
        'delete': 'destroy'  
    })),  
    path('cars', CarViewSet.as_view({  
        'get': 'list',  
        'post': 'create',  
    })),  
    path('cars/<str:pk>', CarViewSet.as_view({  
        'get': 'retrieve',  
        'put': 'update',  
        'delete': 'destroy'  
    })),  
    path('customers', CustomerViewSet.as_view({
```

					КПІ.IT-7311.045440.03.13	Арк.
						5
Змін.	Арк.	№ докум.	Підп.	Дата.		

```

        'get': 'list',
        'post': 'create',
    })),
    path('customers/<str:pk>', CustomerViewSet.as_view({
        'get': 'retrieve',
        'put': 'update',
        'delete': 'destroy'
    })))
]

```

Файл consumer.py:

```

import pika
import json

from main import *

params =
pika.URLParameters('amqps://guomtwqy:0_BmGYhmT09RXj3IcxXwpULhI0_KAZ4V@sparrow.rmq.cloudamqp.
com/guomtwqy')

connection = pika.BlockingConnection(params)

channel = connection.channel()

channel.queue_declare(queue='main')

def callback(ch, method, properties, body):
    print('Received in main')
    data = json.loads(body)
    print(data)

    if properties.content_type == 'employee_created':
        employee = Employee(id=data['id'],
                             employee_full_name=data['employee_full_name'],
                             employee_age=data['employee_age'],
                             employee_sex=data['employee_sex'],
                             employee_adress=data['employee_adress'],
                             employee_phone=data['employee_phone'],
                             employee_passport_data=data['employee_passport_data'],
                             position_code=data['position_code'])
        db.session.add(employee)
        db.session.commit()
        print('Employee Created')

    elif properties.content_type == 'employee_updated':
        employee = Employee.query.get(data['id'])
        employee.employee_full_name = data['employee_full_name']
        employee.employee_age = data['employee_age']
        employee.employee_sex = data['employee_sex']
        employee.employee_adress = data['employee_adress']
        employee.employee_phone = data['employee_phone']
        employee.employee_passport_data = data['employee_passport_data']
        employee.position_code = data['position_code']
        db.session.commit()
        print('Employee Updated')

    elif properties.content_type == 'employee_deleted':
        employee = Employee.query.get(data)
        db.session.delete(employee)
        db.session.commit()
        print('Employee Deleted')

```

					КПІ.IT-7311.045440.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		6

```

elif properties.content_type == 'position_created':
    position = Position(id=data['id'],
                        position_name=data['position_name'],
                        position_salary=data['position_salary'],
                        position_duties=data['position_duties'],
                        position_requirements=data['position_requirements'])
    db.session.add(position)
    db.session.commit()
    print('Position Created')

elif properties.content_type == 'position_updated':
    position = Position.query.get(data['id'])
    position.position_name = data['position_name']
    position.position_salary = data['position_salary']
    position.position_duties = data['position_duties']
    position.position_requirements = data['position_requirements']
    db.session.commit()
    print('Position Updated')

elif properties.content_type == 'position_deleted':
    position = Position.query.get(data)
    db.session.delete(position)
    db.session.commit()
    print('Position Deleted')

elif properties.content_type == 'maker_created':
    maker = Maker(id=data['id'],
                  maker_name=data['maker_name'],
                  maker_country=data['maker_country'],
                  maker_adress=data['maker_adress'],
                  maker_description=data['maker_description'],
                  employee_code=data['employee_code'])
    db.session.add(maker)
    db.session.commit()
    print('Maker Created')

elif properties.content_type == 'maker_updated':
    maker = Maker.query.get(data['id'])
    maker.maker_name = data['maker_name']
    maker.maker_country = data['maker_country']
    maker.maker_adress = data['maker_adress']
    maker.maker_description = data['maker_description']
    maker.employee_code = data['employee_code']
    db.session.commit()
    print('Maker Updated')

elif properties.content_type == 'maker_deleted':
    maker = Maker.query.get(data)
    db.session.delete(maker)
    db.session.commit()
    print('Maker Deleted')

elif properties.content_type == 'additional_equipment_created':
    additional_equipment = AdditionalEquipment(id=data['id'],
                                                equipment_name=data['equipment_name'],
                                                equipment_price=data['equipment_price'],
                                                equipment_characteristics=data['equipment_characteristics'])
    db.session.add(additional_equipment)
    db.session.commit()
    print('AdditionalEquipment Created')

elif properties.content_type == 'additional_equipment_updated':

```

```

additional_equipment = AdditionalEquipment.query.get(data['id'])
additional_equipment.equipment_name = data['equipment_name']
additional_equipment.equipment_price = data['equipment_price']
additional_equipment.equipment_characteristics = data['equipment_characteristics']
db.session.commit()
print('AdditionalEquipment Updated')

elif properties.content_type == 'additional_equipment_deleted':
    additional_equipment = AdditionalEquipment.query.get(data)
    db.session.delete(additional_equipment)
    db.session.commit()
    print('AdditionalEquipment Deleted')

elif properties.content_type == 'body_type_created':
    body_type = BodyType(id=data['id'],
                        body_type_name=data['body_type_name'],
                        body_type_description=data['body_type_description'])
    db.session.add(body_type)
    db.session.commit()
    print('BodyType Created')

elif properties.content_type == 'body_type_updated':
    body_type = AdditionalEquipment.query.get(data['id'])
    body_type.body_type_name = data['body_type_name']
    body_type.body_type_description = data['body_type_description']
    db.session.commit()
    print('BodyType Updated')

elif properties.content_type == 'body_type_deleted':
    body_type = AdditionalEquipment.query.get(data)
    db.session.delete(body_type)
    db.session.commit()
    print('BodyType Deleted')

elif properties.content_type == 'car_created':
    car = Car(id=data['id'],
            car_mark=data['car_mark'],
            maker_code=data['maker_code'],
            body_type_code=data['body_type_code'],
            car_date_of_production=data['car_date_of_production'],
            car_color=data['car_color'],
            car_body_type_number=data['car_body_type_number'],
            car_engine_number=data['car_engine_number'],
            car_description=data['car_description'],
            equipment_code_1=data['equipment_code_1'],
            equipment_code_2=data['equipment_code_2'],
            equipment_code_3=data['equipment_code_3'],
            car_price=data['car_price'],
            employee_code=data['employee_code'])
    db.session.add(car)
    db.session.commit()
    print('Car Created')

elif properties.content_type == 'car_updated':
    car = Car.query.get(data['id'])
    car.car_mark = data['car_mark']
    car.maker_code = data['maker_code']
    car.body_type_code = data['body_type_code']
    car.car_date_of_production = data['car_date_of_production']
    car.car_color = data['car_color']
    car.car_body_type_number = data['car_body_type_number']
    car.car_engine_number = data['car_engine_number']

```

```

car.car_description = data['car_description']
car.equipment_code_1 = data['equipment_code_1']
car.equipment_code_2 = data['equipment_code_2']
car.equipment_code_3 = data['equipment_code_3']
car.car_price = data['car_price']
car.employee_code = data['employee_code']
db.session.commit()
print('Car Updated')

elif properties.content_type == 'car_deleted':
    car = Car.query.get(data)
    db.session.delete(car)
    db.session.commit()
    print('Car Deleted')

elif properties.content_type == 'customer_created':
    customer = Customer(id=data['id'],
                        customer_full_name=data['customer_full_name'],
                        customer_adress=data['customer_adress'],
                        customer_phone=data['customer_phone'],
                        customer_passport_data=data['customer_passport_data'],
                        car_code=data['car_code'],
                        customer_order_data=data['customer_order_data'],
                        customer_data_of_sale=data['customer_data_of_sale'],
                        customer_mark_of_performance=data['customer_mark_of_performance'],
                        customer_mark_of_payment=data['customer_mark_of_payment'],
                        customer_prepayment_percentage=data['customer_prepayment_percentage'],
                        employee_code=data['employee_code'],)
    db.session.add(customer)
    db.session.commit()
    print('Customer Created')

elif properties.content_type == 'customer_updated':
    customer = Customer.query.get(data['id'])
    customer.customer_full_name = data['customer_full_name']
    customer.customer_adress = data['customer_adress']
    customer.customer_phone = data['customer_phone']
    customer.customer_passport_data = data['customer_passport_data']
    customer.car_code = data['car_code']
    customer.customer_mark_of_performance = data['customer_mark_of_performance']
    customer.customer_mark_of_payment = data['customer_mark_of_payment']
    customer.customer_prepayment_percentage = data['customer_prepayment_percentage']
    customer.employee_code = data['employee_code']
    db.session.commit()
    print('Customer Updated')

elif properties.content_type == 'customer_deleted':
    customer = Customer.query.get(data)
    db.session.delete(customer)
    db.session.commit()
    print('Customer Deleted')

channel.basic_consume(queue='main', on_message_callback=callback, auto_ack=True)

print('Started Consuming')

channel.start_consuming()

channel.close()

```


Файл main.py:

```
import requests
from flask import Flask, jsonify
from flask_cors import CORS
from flask_sqlalchemy import SQLAlchemy
from dataclasses import dataclass

app = Flask(__name__)
app.config["SQLALCHEMY_DATABASE_URI"] = 'mysql://root:root@db/main'
CORS(app)

db = SQLAlchemy(app)

@dataclass
class Employee(db.Model):
    id: int
    employee_full_name: str
    employee_age: int
    employee_sex: str
    employee_adress: str
    employee_phone: str
    employee_passport_data: str
    position_code: int

    id = db.Column(db.Integer, primary_key=True, autoincrement=False)
    employee_full_name = db.Column(db.String(50))
    employee_age = db.Column(db.Integer)
    employee_sex = db.Column(db.String(1))
    employee_adress = db.Column(db.String(150))
    employee_phone = db.Column(db.String(50))
    employee_passport_data = db.Column(db.String(50))
    position_code = db.Column(db.Integer, db.ForeignKey('position.id'), nullable=False)
    makers = db.relationship('Maker', backref='employee', lazy=True)
    cars = db.relationship('Car', backref='employee', lazy=True)
    customers = db.relationship('Customer', backref='employee', lazy=True)

@dataclass
class Position(db.Model):
    id: int
    position_name: str
    position_salary: int
    position_duties: str
    position_requirements: str

    id = db.Column(db.Integer, primary_key=True, autoincrement=False)
    position_name = db.Column(db.String(50))
    position_salary = db.Column(db.Integer)
    position_duties = db.Column(db.String(200))
    position_requirements = db.Column(db.String(200))
    employees = db.relationship('Employee', backref='position', lazy=True)

@dataclass
class Maker(db.Model):
    id: int
    maker_name: str
    maker_country: str
    maker_adress: str
    maker_description: str
    employee_code: int

    id = db.Column(db.Integer, primary_key=True, autoincrement=False)
```

					КПІ.IT-7311.045440.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		10

```

maker_name = db.Column(db.String(50))
maker_country = db.Column(db.String(50))
maker_adress = db.Column(db.String(150))
maker_description = db.Column(db.Text)
employee_code = db.Column(db.Integer, db.ForeignKey('employee.id'), nullable=False)
cars = db.relationship('Car', backref='maker', lazy=True)

```

```

@dataclass
class AdditionalEquipment(db.Model):
    id: int
    equipment_name: str
    equipment_characteristics: str
    equipment_price: int

    id = db.Column(db.Integer, primary_key=True, autoincrement=False)
    equipment_name = db.Column(db.String(50))
    equipment_characteristics = db.Column(db.Text)
    equipment_price = db.Column(db.Integer)

```

```

@dataclass
class BodyType(db.Model):
    id: int
    body_type_name: str
    body_type_description: str

    id = db.Column(db.Integer, primary_key=True, autoincrement=False)
    body_type_name = db.Column(db.String(50))
    body_type_description = db.Column(db.Text)
    cars = db.relationship('Car', backref='bodyType', lazy=True)

```

```

@dataclass
class Car(db.Model):
    id: int
    car_mark: str
    maker_code: int
    body_type_code: int
    car_date_of_production: str
    car_color: str
    car_body_type_number: int
    car_engine_number: int
    car_description: str
    equipment_code_1: int
    equipment_code_2: int
    equipment_code_3: int
    car_price: int
    employee_code: int

    id = db.Column(db.Integer, primary_key=True, autoincrement=False)
    car_mark = db.Column(db.String(50))
    maker_code = db.Column(db.Integer, db.ForeignKey('maker.id'), nullable=False)
    body_type_code = db.Column(db.Integer, db.ForeignKey('body_type.id'), nullable=False)
    car_date_of_production = db.Column(db.Date)
    car_color = db.Column(db.String(50))
    car_body_type_number = db.Column(db.Integer)
    car_engine_number = db.Column(db.Integer)
    car_description = db.Column(db.Text)
    equipment_code_1 = db.Column(db.Integer)
    equipment_code_2 = db.Column(db.Integer)
    equipment_code_3 = db.Column(db.Integer)
    car_price = db.Column(db.Integer)
    employee_code = db.Column(db.Integer, db.ForeignKey('employee.id'), nullable=False)
    customer = db.relationship("Customer", uselist=False, backref="car")

```

```

@dataclass
class Customer(db.Model):
    id: int
    customer_full_name: str
    customer_adress: str
    customer_phone: str
    customer_passport_data: str
    car_code: int
    customer_order_data: str
    customer_data_of_sale: str
    customer_mark_of_performance: bool
    customer_mark_of_payment: bool
    customer_prepayment_percentage: int
    employee_code: int

    id = db.Column(db.Integer, primary_key=True, autoincrement=False)
    customer_full_name = db.Column(db.String(50))
    customer_adress = db.Column(db.String(150))
    customer_phone = db.Column(db.String(50))
    customer_passport_data = db.Column(db.String(200))
    car_code = db.Column(db.Integer, db.ForeignKey('car.id'), nullable=False)
    customer_order_data = db.Column(db.Date)
    customer_data_of_sale = db.Column(db.Date)
    customer_mark_of_performance = db.Column(db.Boolean)
    customer_mark_of_payment = db.Column(db.Boolean)
    customer_prepayment_percentage = db.Column(db.Integer)
    employee_code = db.Column(db.Integer, db.ForeignKey('employee.id'), nullable=False)

@app.route('/api/employees', methods=['GET'])
def all_employees():
    return jsonify(Employee.query.all())
@app.route('/api/employees/<int:id>', methods=['GET'])
def one_employee(id):
    return jsonify(Employee.query.get(id))
@app.route('/api/positions', methods=['GET'])
def all_positions():
    return jsonify(Position.query.all())
@app.route('/api/positions/<int:id>', methods=['GET'])
def one_position(id):
    return jsonify(Position.query.get(id))
@app.route('/api/makers', methods=['GET'])
def all_makers():
    return jsonify(Maker.query.all())
@app.route('/api/makers/<int:id>', methods=['GET'])
def one_maker(id):
    return jsonify(Maker.query.get(id))
@app.route('/api/additional equipments', methods=['GET'])
def all equipments():
    return jsonify(AdditionalEquipment.query.all())
@app.route('/api/additional equipments/<int:id>', methods=['GET'])
def one_equipment(id):
    return jsonify(AdditionalEquipment.query.get(id))
@app.route('/api/body_types', methods=['GET'])
def all_body_types():
    return jsonify(BodyType.query.all())
@app.route('/api/body_types/<int:id>', methods=['GET'])
def one_body_type(id):
    return jsonify(BodyType.query.get(id))
@app.route('/api/cars', methods=['GET'])
def all_cars():
    return jsonify(Car.query.all())
@app.route('/api/cars/<int:id>', methods=['GET'])

```

					КПІ.IT-7311.045440.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		12

```

def one_car(id):
    return jsonify(Car.query.get(id))
@app.route('/api/customers', methods=['GET'])
def all_customers():
    return jsonify(Customer.query.all())
@app.route('/api/customers/<int:id>', methods=['GET'])
def one_customer(id):
    return jsonify(Customer.query.get(id))
if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0')
Файл menu.tsx:
import React from 'react';
const Menu = () => {
    return (
        <nav id="sidebarMenu" className="col-md-3 col-lg-2 d-md-block bg-light sidebar collapse">
            <div className="sidebar-sticky pt-3">
                <ul className="nav flex-column">
                    <li className="nav-item">
                        <a className="nav-link active" href="/admin/employees">
                            Employees
                        </a>
                        <a className="nav-link active" href="/admin/employees">
                            Positions
                        </a>
                        <a className="nav-link active" href="/admin/employees">
                            Makers
                        </a>
                        <a className="nav-link active" href="/admin/employees">
                            Additional Equipment
                        </a>
                        <a className="nav-link active" href="/admin/employees">
                            Body Types
                        </a>
                        <a className="nav-link active" href="/admin/employees">
                            Cars
                        </a>
                        <a className="nav-link active" href="/admin/employees">
                            Customers
                        </a>
                    </li>
                </ul>
            </div>
        </nav>
    );
};
export default Menu;

```

Файл nav.tsx:

```

import React from 'react';
const Nav = () => {
    return (
        <nav className="navbar navbar-dark sticky-top bg-dark flex-md-nowrap p-0 shadow">
            <a className="navbar-brand col-md-3 col-lg-2 mr-0 px-3" href="#">Автосалон</a>
            <button className="navbar-toggler position-absolute d-md-none collapsed" type="button"
                data-toggle="collapse"
                data-target="#sidebarMenu" aria-controls="sidebarMenu" aria-expanded="false"
                aria-label="Toggle navigation">
                <span className="navbar-toggler-icon"></span>
            </button>
            <ul className="navbar-nav px-3">
                <li className="nav-item text-nowrap">
                    <a className="nav-link" href="#">Sign out</a>
                </li>
            </ul>
        </nav>
    );
};

```

```

        </li>
      </ul>
    </nav>
  );
};

```

export default Nav;

Файл wrapper.tsx:

```

import React, {PropsWithChildren} from 'react';
import Nav from "../components/Nav";
import Menu from "../components/Menu";

```

```

const Wrapper = (props: PropsWithChildren<any>) => {
  return (
    <div>
      <Nav/>
      <div className="container-fluid">
        <div className="row">

          <Menu/>
          <main role="main" className="col-md-9 ml-sm-auto col-lg-10 px-md-4">
            {props.children}
          </main>
        </div>
      </div>
    </div>
  );
};
export default Wrapper;

```

Файл employees.tsx:

```

import React, {useEffect, useState} from 'react';
import Wrapper from "../Wrapper";
import {Employee} from "../interfaces/employee";
import {Link} from "react-router-dom";

```

```

const Employees = () => {
  const [employees, setEmployees] = useState([]);

  useEffect(() => {
    (
      async () => {
        const response = await fetch('http://localhost:8000/api/employees');

        const data = await response.json();

        setEmployees(data);
      }
    )();
  }, []);

  const del = async (id: number) => {
    if (window.confirm('Are you sure you want to delete this employee?')) {
      await fetch(`http://localhost:8000/api/employees/${id}`, {
        method: 'DELETE'
      });
      setEmployees(employees.filter((p: Employee) => p.id !== id));
    }
  }

  return (
    <Wrapper>

```

					КПІ.IT-7311.045440.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		14

```

<div className="pt-3 pb-2 mb-3 border-bottom">
  <div className="btn-toolbar mb-2 mb-md-0">
    <Link to="/admin/employees/create" className="btn btn-sm btn-outline-secondary">Add</Link>
  </div>
</div>
<div className="table-responsive">
  <table className="table table-striped table-sm">
    <thead>
      <tr>
        <th>#</th>
        <th>ПІБ</th>
        <th>Вік</th>
        <th>Стать</th>
        <th>Адреса</th>
        <th>Номер телефону</th>
        <th>Паспортні дані</th>
        <th>Посада</th>
        <th></th>
      </tr>
    </thead>
    <tbody>
      {employees.map(
        (e: Employee) => {
          return (
            <tr key={e.id}>
              <td>{e.id}</td>
              <td>{e.employee_full_name}</td>
              <td>{e.employee_age}</td>
              <td>{e.employee_sex}</td>
              <td>{e.employee_adress}</td>
              <td>{e.employee_phone}</td>
              <td>{e.employee_passport_data}</td>
              <td>{e.position_code}</td>
              <td>
                <div className="btn-group mr-2">
                  <Link to={`/admin/employees/${e.id}/edit`}
                    className="btn btn-sm btn-outline-secondary">Edit</Link>
                  <a href="#" className="btn btn-sm btn-outline-secondary"
                    onClick={() => del(e.id)}>Delete</a>
                </div>
              </td>
            </tr>
          )
        })}
    </tbody>
  </table>
</div>
</Wrapper>
);
};

export default Employees

```

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2022 р.

ВЕБЗАСТОСУНОК ДЛЯ УПРАВЛІННЯ АВТОСАЛОНОМ

Програма та методика тестування

КПІ.IT-7311.045440.04.51

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Олена ХАЛУС

Нормоконтроль:

_____ Катерина ЛІЩУК

Виконавець:

_____ Олександр ПАНЬКІВ

Київ – 2022

ЗМІСТ

1 Об'єкт випробувань	3
2 Мета тестування	4
3 Методи тестування.....	5
4 Засоби та порядок тестування.....	6

					КПІ.ІТ-7311.045440.04.51	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		2

1 ОБ'ЄКТ ВИПРОБУВАНЬ

Вебзастосунок для управління автосалоном складається з кількох компонентів:

- Django-сервіс для адміністрування;
- Flask-сервіс для перегляду;
- RabbitMQ для обміну повідомленнями між цими двома сервісами;
- React-клієнт для взаємодії користувача з системою.

Клієнт взаємодіє з сервісами за допомогою REST API.

					КПІ.IT-7311.045440.04.51	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		3

2 МЕТА ТЕСТУВАННЯ

Метою тестування є наступне:

- перевірка коректності виконання бажаних функцій компонентами системи;
- знаходження проблем та недоліків з метою їх усунення;
- перевірка можливості роботи з графічним інтерфейсом.

					КПІ.ІТ-7311.045440.04.51	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		4

3 МЕТОДИ ТЕСТУВАННЯ

Для тестування програмного забезпечення використовуються такі методи:

- модульне тестування для перевірки коректності роботи окремих модулів;
- інтеграційне тестування, з допомогою якого перевіряється робота кількох модулів в системі разом;
- тестування API для виявлення коректності адресування та обробки REST-запитів сервісами;
- ручне тестування, що дозволить виявити проблеми, які можуть виникнути в процесі взаємодії з користувацьким інтерфейсом.

					КПІ.IT-7311.045440.04.51	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		5

4 ЗАСОБИ ТА ПОРЯДОК ТЕСТУВАННЯ

Модульне тестування виконується за допомогою «TestCase» із «django.test». Тестування API проводиться з використанням програмного засобу «Postman». Ручне тестування виконується як взаємодія тестувальника з користувацьким інтерфейсом. При цьому ж тестування API та ручне тестування водночас є інтеграційним, адже задіюються одразу кілька модулів вобзастосунку. RabbitMQ Management дозволить протестувати доставку повідомлень.

Порядок тестування є наступним:

- запуск написаних unit-тестів;
- відправка REST-запитів до сервісів за допомогою Postman;
- перевірка коректності зміни даних в БД при виконанні запитів PUT, POST, DELETE;
- перевірка доставки повідомлень;
- ручне тестування користувацького інтерфейсу.

					КПІ.IT-7311.045440.04.51	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		6

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2022 р.

ВЕБЗАСТОСУНОК ДЛЯ УПРАВЛІННЯ АВТОСАЛОНОМ

Керівництво користувача

КПІ.IT-7311.045440.05.34

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Олена ХАЛУС

Нормоконтроль:

_____ Катерина ЛІЩУК

Виконавець:

_____ Олександр ПАНЬКІВ

Київ – 2022

ЗМІСТ

1 Загальні відомості	3
2 Підготовка до роботи.....	4
2.1 Системні вимоги для коректної роботи	4
2.2 Завантаження застосунку.....	4
2.3 Перевірка коректної роботи.....	4
3 Робота із застосунком.....	5

					КПІ.ІТ-7311.045440.05.34	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		2

1 ЗАГАЛЬНІ ВІДОМОСТІ

«Вебзастосунок для управління автосалоном» - це програмний продукт, призначений для управління автосалоном. Він дозволяє отримувати, додавати, змінювати та видаляти дані про:

- працівників;
- автомобілі;
- замовників;
- посади;
- типи кузовів;
- виробників;
- додаткове обладнання для автомобілів.

Він заснований на архітектурі мікросервісів, деякі компоненти є слабо зв'язаними. Є два основних сервіси:

- admin відповідає за можливість вносити зміни в БД;
- main надає можливість тільки переглядати дані в БД.

Вони обидва надають REST API для взаємодії. Клієнт, розроблений з використанням React, зв'язується з кінцевими точками обох сервісів. Доступ до цих компонентів здійснюється через різні URI.

					КПІ.IT-7311.045440.05.34	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		3

2 ПІДГОТОВКА ДО РОБОТИ

2.1 Системні вимоги для коректної роботи

Для успішної роботи даного застосунку необхідне виконання наступних вимог:

- наявність персонального комп'ютера під управлінням операційної системи Windows (бажано Windows 10);
- наявність доступу до Інтернету для використання стилів сторінок клієнта та підключення до хмарного RabbitMQ;
- встановлені програмні засоби Docker Desktop, npm та python;
- наявність не менш, ніж: 8 ГБ оперативної пам'яті, 5 ГБ вільного дискового простору та процесора intel core i3 третього покоління або аналога.

2.2 Завантаження застосунку

Розгортання застосунку можна розпочинати після завантаження папок з розробленими компонентами на комп'ютер. Спершу необхідно обрати план сервісу Cloud AMQP та скопіювати URI, за яким встановлюватиметься з'єднання з брокером повідомлень. Цю адресу необхідно вписати в «pika.URLParameters(`\*скопійований URI\*`)» у файлах «consumer.py» та «producer.py» у директорії «admin», а також у файлі «consumer.py» в директорії «main».

Після зазначених дій у директоріях «admin» та «main» відкривається консоль та вводиться команда «docker-compose up». Після того, як завершиться збірка і розгортання контейнерів Docker, в директорії «front/react_crud» виконується команда «npm start», що запускає вебсервер react-клієнта.

					КПІ.IT-7311.045440.05.34	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		4

2.3 Перевірка коректної роботи

По завершенню розгортання компонентів у Docker Desktop повинні з'явитися нові контейнери (рисуюнок 2.1).



Рисуюнок 2.1 – Активні Docker-контейнери

В консолі, що в директорії «react-crud», в свою чергу, повинен з'явитися запис «Project running at... compiled...», що свідчить про успішний запуск.

3 РОБОТА ІЗ ЗАСТОСУНКОМ

При запуску react-клієнта повинен відкритися браузер та сторінка з посиланням «<http://localhost:3000>». Це основна частина вебзастосунку. Можна переглянути працівників із БД (рисунок 3.1).

— Вік: 21 — Стать: M — Адреса: m. Kyiv, Klovskа, 13a — Номер телефону: 097-777-77-77 — Паспортні дані: 777777777 — Посада: 5 Kozlov Anton Artemovych	— Вік: 57 — Стать: W — Адреса: m. Kyiv, Kashtanova, 36b — Номер телефону: 067-893-19-28 — Паспортні дані: AK4774 — Посада: 3 Dmytriieva Antonina Stepanivna	— Вік: 18 — Стать: M — Адреса: m. Kyiv, Ivana Mazepy, 11b — Номер телефону: 096-666-66-66 — Паспортні дані: 6969696969 — Посада: 1 Mozol Pavlo Yuriiiovych
— Вік: 28 — Стать: M — Адреса: m. Bila Tserkva, Lesi Ukrainky, 15 — Номер телефону: 098-305-35-45 — Паспортні дані: AT8932 — Посада: 2 Kovalchuk Vitalii Danylovych	— Вік: 24 — Стать: M — Адреса: m. Kyiv, Bandery, 14a — Номер телефону: 098-800-90-70 — Паспортні дані: 7821353204 — Посада: 4 Ilkiv Stanislav Oleksandrovych	— Вік: 29 — Стать: M — Адреса: m. Kyiv, Dzerzhynskoho, 23b — Номер телефону: 096-432-36-32 — Паспортні дані: NK2152 — Посада: 2 Sekret Viktoriia Marianovych

Рисунок 3.1 – Виведення списку співробітників

За потреби можна отримати детальну інформацію про посаду, натиснувши на відповідне посилання (рисунок 3.2).

— Код: 1
— Назва посади: Menedzher
— Обов'язки: Upravlinnia personalom
— Вимоги: Liderski yakosti
— Зарплата: 90000

Рисунок 3.2 – Інформація про посаду

Перейшовши за адресою «<http://localhost:3000/admin/employees>», можна отримати список тих же співробітників, але у вигляді таблиці та з можливістю редагування. Це адміністративна частина застосунка (рисунок 3.3).

Автосалон

Sign out

Employees

Positions

Makers

Additional Equipment

Body Types

Cars

Customers

Add

#	ПІБ	Вік	Стать	Адреса	Номер телефону	Паспортні дані	Посада		
1	Kozlov Anton Artemovych	21	M	m. Kyiv, Klovsk, 13a	097-777-77-77	777777777	5	Edit	Delete
2	Dmytriieva Antonina Stepanivna	57	W	m. Kyiv, Kashtanova, 36b	067-893-19-28	AK4774	3	Edit	Delete
3	Mozol Pavlo Yuriiovych	18	M	m. Kyiv, Ivana Mazepy, 11b	096-666-66-66	6969696969	1	Edit	Delete
4	Kovalchuk Vitalii Danylovych	28	M	m. Bila Tserkva, Lesi Ukrainky, 15	098-305-35-45	AT8932	2	Edit	Delete
5	Ilkiv Stanislav Oleksandrovych	24	M	m. Kyiv, Bandery, 14a	098-800-90-70	7821353204	4	Edit	Delete
6	Sekret Viktoriia Marianovych	29	M	m. Kyiv, Dzerzhynskoho, 23b	096-432-36-32	NK2152	2	Edit	Delete
7	Davchenko Anatolii Zakharovych	44	M	m. Kyiv, Metalistiv, 16	097-123-33-23	AK2337	3	Edit	Delete
8	Enhelhart Pavlo Dmytrovych	21	M	m. Kyiv, Shevchenka, 37	097-148-08-13	9250386483	2	Edit	Delete
9	Synkiv Roksolana Vasylivna	24	M	m. Kyiv, Kashtanova, 32a	098-224-64-32	AT3242	4	Edit	Delete

Рисунок 3.3 – Список співробітників з можливістю внесення змін

Зліва знаходиться навігаційне меню для перегляду різних даних БД. Над списком знаходиться кнопка «Add» для додавання нових співробітників. Навпроти кожного запису є кнопки «Edit» та «Delete» для внесення змін та видалення відповідно.

Для прикладу, при натисканні «Add» відкривається форма з порожніми полями (рисунок 3.4).

Автосалон	Sign out
Employees Positions Makers Additional Equipment Body Types Cars Customers	ПІБ Вік Стать Адреса Номер телефону Паспортні дані Посада Зберегти

Рисунок 3.4 – Додавання співробітника

Після заповнення відповідних елементів «input» та відправки форми буде створено нового співробітника. Новий співробітник не буде створений, якщо один з атрибутів порожній.

Також, за потреби, можна зайти до власної адмін-панелі сервісу Django за писиланням «<http://localhost:8000/admin/carshowroom/>» (рисунок 3.5).

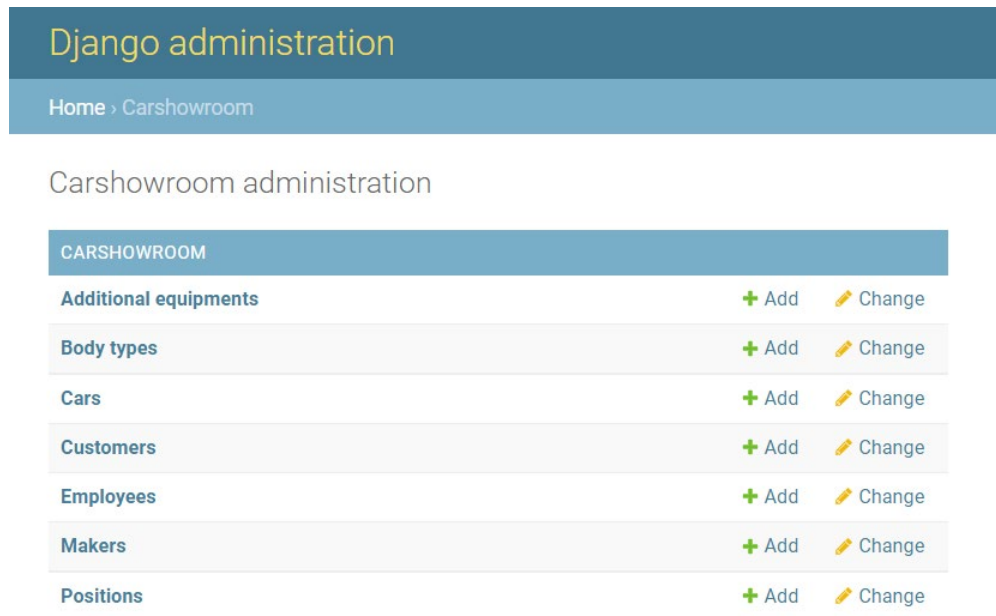


Рисунок 3.5 – Адміністративна панель Django

Проте, якщо вносити зміни в цій панелі, вони не будуть узгоджені із базою даних main. Логін та пароль цієї панелі задає адміністратор системи.

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2022 р.

ВЕБЗАСТОСУНОК ДЛЯ УПРАВЛІННЯ АВТОСАЛОНОМ

Графічний матеріал

КПІ.IT-7311.045440.06.99

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Олена ХАЛУС

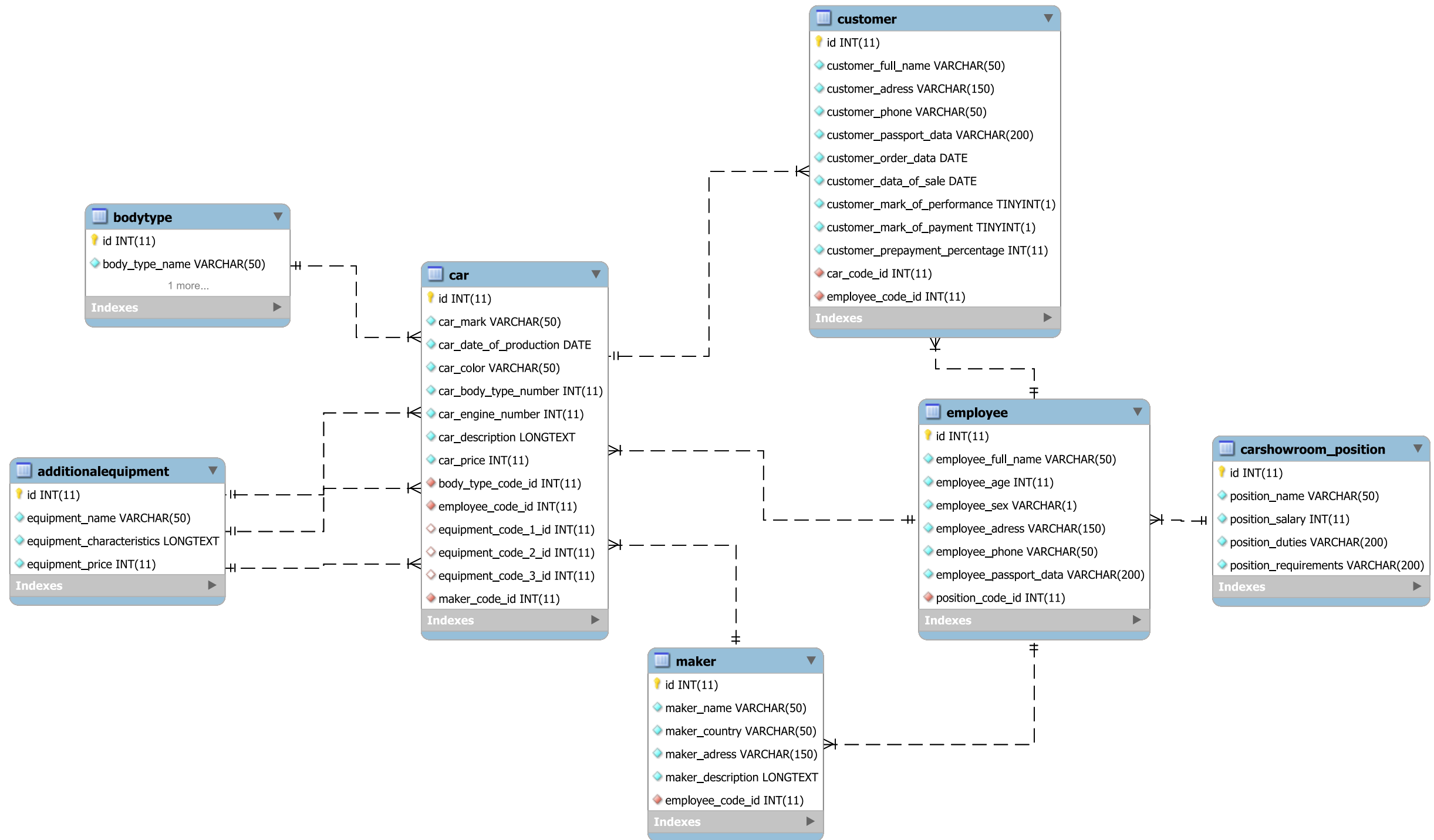
Нормоконтроль:

_____ Катерина ЛІЩУК

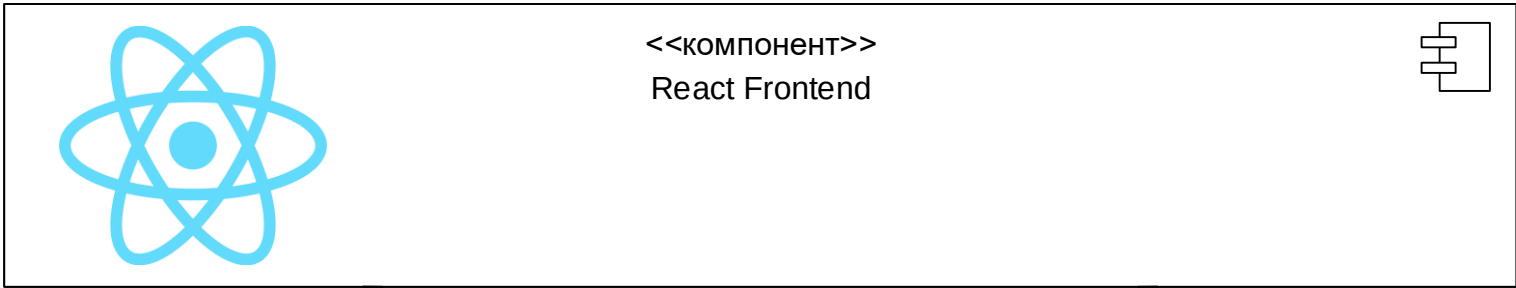
Виконавець:

_____ Олександр ПАНЬКІВ

Київ – 2022



					КПІ.ІТ-7311.045440.06.99СБД						
					Схема бази даних	Літера			Маса	Масштаб	
Зм.	Арк.	№ документа	Підпис	Дата							
Розробив	Паньків О.В.										
Перевірів	Халус О.А.										
Т. кон.						Аркуш			Аркушів		
					Вебзастосунок для управління автосалоном	КПІ ім.Ігоря Сікорського Кафедра ІПІ гр. ІТ-81					
Н. кон.	Ліщук К.І.										
Затвердив	Халус О.А.										



					КПІ.ІТ-7311.045440.07.99СС
					Схема структурна компонентів програмного забезпечення Літера Маса Масштаб
Зм.	Арк.	№ документа	Підпис	Дата	
Розробив		Паньків О.В.			
Перевірив		Халус О.А.			
Т. кон.					Аркуш Аркушів
					КПІ ім.Ігоря Сікорського Кафедра ІПІ гр. ІТ-81
Н. кон.		Ліщук К.І.			
Затвердив		Халус О.А.			

Автосалон		Sign out																					
Employees	Add																						
Positions																							
Makers																							
Additional Equipment																							
Body Types																							
Cars																							
Customers																							

Django administration

Home / Carshowroom / Cars

Authentication and Authorization

Groups + Add

Users + Add

Carshowroom

Additional equipments + Add

Body types + Add

Cars + Add

Customers + Add

Employees + Add

Makers + Add

Positions + Add

Select car to change

Action: Go

0 of 10 selected

☐	ID	МАРКА	ВИРОБНИК	ТИП КУЗОВА	ДАТА ВИРОБНИЦТВА	КОЛІР
☐	10	SX4 1.6 MT 4WD GL	Suzuki	Miniven	Jan. 1, 2020	Siriyi
☐	9	SX4 1.6 MT 4WD GL	Suzuki	Khetchbek	Jan. 1, 2020	Synii
☐	8	Rifter	Peugeot	Khetchbek	Jan. 1, 2020	Siriyi
☐	7	3008	Peugeot	Pozashliakhovyy (krossover)	Jan. 1, 2020	Bilyi
☐	6	Scala	Škoda	Khetchbek	Jan. 1, 2020	Chornyi
☐	5	Fabla 1.2 TSI	Škoda	Sedan	Jan. 1, 2015	Bilyi
☐	4	X6	BMW	Khetchbek	Jan. 1, 2008	Chervonyi
☐	3	530 xDrive	BMW	Universal	Jan. 1, 2013	Chornyi
☐	2	A4 quattro IDEAL	Audi	Sedan	Jan. 1, 2011	Bilyi
☐	1	A6	Audi	Sedan	Jan. 1, 2014	Siriyi

10 cars

ADD CAR +

FILTER

Vu Тип кузова

All

Sedan

Universal

Khetchbek

Miniven

Pozashliakhovyy (krossover)

Vu Виробник

All

Audi

BMW

Škoda

Suzuki

Peugeot

- **Вік:** 21
- **Стать:** М
- **Адреса:** м. Kyiv, Klovska, 13a
- **Номер телефону:** 097-777-77-77
- **Паспортні дані:** 777777777
- **Посада:** 5

Kozlov Anton Artemovych

- **Вік:** 28
- **Стать:** М
- **Адреса:** м. Bila Tserkva, Lesi Ukrainky, 15
- **Номер телефону:** 098-305-35-45
- **Паспортні дані:** AT8932
- **Посада:** 2

Kovalchuk Vitalii Danylovych

- **Вік:** 44
- **Стать:** М

- **Вік:** 57
- **Стать:** W
- **Адреса:** m. Kyiv, Kashtanova, 36b
- **Номер телефону:** 067-893-19-28
- **Паспортні дані:** AK4774
- **Посада:** 3

Dmytriieva Antonina Stepanivna

- **Вік:** 24
- **Стать:** M
- **Адреса:** m. Kyiv, Bandery, 14a
- **Номер телефону:** 098-800-90-70
- **Паспортні дані:** 7821353204
- **Посада:** 4

Ilkiv Stanislav Oleksandrovych

- **Вік:** 21
- **Стать:** M

- **Вік:** 18
- **Стать:** М
- **Адреса:** м. Kyiv, Ivana Mazepy, 11b
- **Номер телефону:** 096-666-66-66
- **Паспортні дані:** 6969696969
- **Посада:** 1

Mozol Pavlo Yuriiovych

- **Вік:** 29
- **Стать:** М
- **Адреса:** м. Kyiv, Dzerzhynskoho, 23b
- **Номер телефону:** 096-432-36-32
- **Паспортні дані:** NK2152
- **Посада:** 2

Sekret Viktoriia Marianovych

- **Вік:** 24
- **Стать:** М

- **Код:** 1
- **Назва посади:** Menedzher
- **Обов'язки:** Upravlinnia personalom
- **Вимоги:** Liderski yakosti
- **Зарплата:** 90000

- **Код:** 4
- **Назва посади:** Prodavets-konsultant
- **Обов'язки:** Vmnirna perekonlyvo rozmovliaty
- **Вимоги:** Zmushuie kliientiv kupyty avtomobil maks. kompleksatsii
- **Зарплата:** 8000

- **Код:** 2
- **Назва посади:** Mekhanik
- **Обов'язки:** Remont avtomobiliv
- **Вимоги:** Naiavnist ruk
- **Зарплата:** 30000

- **Код:** 5
- **Назва посади:** Administrator
- **Обов'язки:** Syn vlasnyka avtosalonu
- **Вимоги:** Upravlinnia avtosalonom
- **Зарплата:** 200000

- **Код:** 3
- **Назва посади:** Prybyralnyk
- **Обов'язки:** Vyshcha osvita (mahistr filolohii)
- **Вимоги:** Prybyrannia avtosalonu
- **Зарплата:** 10000

Django administration

Site administration

AUTHENTICATION AND AUTHORIZATION	
Groups	+ Add ✎ Change
Users	+ Add ✎ Change

CARSHOWROOM	
Additional equipments	+ Add ✎ Change
Body types	+ Add ✎ Change
Cars	+ Add ✎ Change
Customers	+ Add ✎ Change
Employees	+ Add ✎ Change
Makers	+ Add ✎ Change
Positions	+ Add ✎ Change

Автосалон	Sign out
Employees	Kozlov Anton Artemovych
Positions	
Makers	
Additional Equipment	1
Body Types	
Cars	M
Customers	
	ПІБ
	Kozlov Anton Artemovych
	Вік
	1
	Стать
	M
	Адреса
	m. Kyiv, Klovska, 13a
	Номер телефону
	097-777-77-77
	Паспортні дані
	777777777
	Посада
	4
	Зберегти

					КПІ.ІТ-7311.045440.08.99КЕ				
					Креслення вигляду екранних форм	Літера	Маса	Масштаб	
Зм.	Арк.	№ документа	Підпис	Дата					
Розробив		Паньків О.В.							
Перевірив		Халус О.А.							
Т. кон.									
						Аркуш	Аркушів		
Н. кон.		Ліщук К.І.			Вебзастосунок для управління автосалоном	КПІ ім.Ігоря Сікорського Кафедра ІПІ гр. ІТ-81			
Затвердив		Халус О.А.							

Имя пользователя:
Лісовиченко Олег Іванович

ID проверки:
1011636540

Дата проверки:
22.06.2022 18:21:46 EEST

Тип проверки:
Doc vs Internet + Library

Дата отчета:
22.06.2022 18:22:14 EEST

ID пользователя:
76913

Название файла: IT-81_Паньків_ПЗ

Количество страниц: 64 Количество слов: 9901 Количество символов: 76141 Размер файла: 1.22 MB ID файла: 1011503430

Обнаружены модификации текста (могут влиять на процент совпадений)

3.47%

Совпадения

Наибольшее совпадение: 0.47% с Интернет-источником (<https://www.codegrepper.com/code-examples/whatever/%3C1..>)

1.59% Источники из Интернета

12

Страница 66

2.67% Источники из Библиотеки

172

Страница 66

0% Цитат

Исключение цитат выключено

Исключение списка библиографических ссылок выключено

0% Исключений

Нет исключенных источников

Модификации

Обнаружены модификации текста. Подробная информация доступна в онлайн-отчете.

Замененные символы

1

Подозрительное форматирование

14
страниц